ESCUELA DE INGENIERÍA INFORMÁTICA



PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO

INFORME 3: ANÁLISIS Y DISEÑO ESTRUCTURADO DEL SISTEMA BEETRACER

Trazabilidad y evidencia fotográfica del desconsolidado en terminales extraportuarios

Benjamín Contreras
Renato Espina
Joaquín Guarda
Ignacio Ruiz
Sergio Codoceo

ICI-3242 Modelamiento de Software

Profesor: Broderick Crawford
Paralelo: 3

Noviembre 2026

ÍNDICE GENERAL

Índice General i

Lista de Figuras iv

Lista de Tablas v

1 Análisis estructurado 1

1 Introducción 2

2 Descripción general 3

3 Modelo ambiental 5

3.1 Declaración de propósitos 5

3.2 Diagrama de contexto 5

3.3 Lista de acontecimientos 7

4 Modelo de comportamiento 10

4.1 Diagramas de flujo de datos 10

4.1.1 DFD de nivel 0 10

4.1.2 DFD de nivel 1 13

4.1.3 DFD de nivel 2 21

4.2 Diagrama entidad-relación 23

4.3 Mini especificaciones de los procesos elementales 25

4.3.1 Proceso 1: Administrar datos maestros 25

4.3.2 Proceso 2: Importar manifiesto 27

4.3.3 Proceso 3: Programar faenas 30
4.3.4 Proceso 4: Ejecutar operación en terreno 32
4.3.5 Proceso 5: Validar reporte 34
4.3.6 Proceso 6: Distribuir y consultar reportes 36
4.4 Diccionario de datos 38
4.4.1 Notación 38
4.4.2 Flujos de datos 38
4.4.3 Almacenes de datos 41
4.4.4 Datos elementales 42
4.4.5 Dominios y valores posibles 44

2 Diseño estructurado 46

5 Diagrama de estructura 47
5.1 Derivación desde los diagramas de flujo de datos 47
5.2 Criterios de calidad del diseño 48
5.3 Diagramas 49

6 Modelo relacional 57
6.1 Esquema relacional 57
6.2 Restricciones de integridad 60

7 Especificaciones de cada módulo 61
7.1 Módulo principal 61
7.2 Subsistema 1: Administrar datos maestros 62
7.3 Subsistema 2: Importar manifiesto 63
7.4 Subsistema 3: Programar faenas 65
7.5 Subsistema 4: Ejecutar operación en terreno 66

7.6 Subsistema 5: Validar reporte 68
7.7 Subsistema 6: Distribuir y consultar reportes 69
7.8 Módulos predefinidos 71

8 Conclusiones **72**

9 Referencias bibliográficas **73**

LISTA DE FIGURAS

1 Diagrama de contexto 7

2 DFD de nivel 0 12

3 DFD de nivel 1 - Proceso 1 15

4 DFD de nivel 1 - Proceso 2 16

5 DFD de nivel 1 - Proceso 3 17

6 DFD de nivel 1 - Proceso 4 18

7 DFD de nivel 1 - Proceso 5 19

8 DFD de nivel 1 - Proceso 6 20

9 DFD de nivel 2 - Proceso 2.3 21

10 DFD de nivel 2 - Proceso 3.2 22

11 DFD de nivel 2 - Proceso 4.3 22

12 Diagrama entidad-relación 24

13 Notación del diagrama de estructura 47

14 Diagrama de estructura - Panorama general 50

15 Diagrama de estructura - Administrar datos maestros 51

16 Diagrama de estructura - Importar manifiesto 52

17 Diagrama de estructura - Programar faenas 53

18 Diagrama de estructura - Ejecutar operación en terreno 54

19 Diagrama de estructura - Validar reporte 55

20 Diagrama de estructura - Distribuir y consultar reportes 56

21 Modelo relacional 59

LISTA DE TABLAS

1 Lista de acontecimientos del sistema BeeTracer. 8

2 Notación del diccionario de datos. 38

3 Diccionario de datos - Flujos de datos. 38

4 Diccionario de datos - Almacenes. 41

5 Diccionario de datos - Datos elementales. 42

6 Restricciones de integridad del modelo relacional. 60

7 Módulos del panorama general. 61

8 Módulos del subsistema Administrar datos maestros. 62

9 Módulos del subsistema Importar manifiesto. 64

10 Módulos del subsistema Programar faenas. 65

11 Módulos del subsistema Ejecutar operación en terreno. 67

12 Módulos del subsistema Validar reporte. 68

13 Módulos del subsistema Distribuir y consultar reportes. 69

14 Módulos predefinidos del diseño. 71

PARTE 1

ANÁLISIS ESTRUCTURADO

1. INTRODUCCIÓN

En el Informe 2 se reconstruyó, mediante ingeniería inversa, el funcionamiento de BeeTracer, la plataforma de Bilix Ingeniería que digitaliza el desconsolidado de contenedores en terminales extraportuarios. Ese trabajo usó el Lenguaje Unificado de Modelado (UML) y el enfoque orientado a objetos de Larman (2003): se identificaron los actores, los casos de uso, las interacciones entre objetos y un diagrama de clases que explica la estructura del sistema.

El presente informe corresponde a la tercera entrega de la asignatura Modelamiento de Software y aborda el mismo sistema desde otro paradigma: el **análisis y diseño estructurado**. En lugar de preguntar qué objetos colaboran, se pregunta qué datos entran al sistema, cómo se transforman y dónde se almacenan. Para ello se sigue el análisis estructurado moderno de Yourdon (1989) y DeMarco (1979), y el diseño estructurado de Yourdon y Constantine (1979) y Page-Jones (1988). Las fuentes de información son las mismas del informe anterior: la charla técnica del equipo de BeeTracer (Equipo de desarrollo de BeeTracer, Bilix Ingeniería, 2026), la información pública de la plataforma (Bilix Ingeniería, s.f.) y el propio modelo UML, que aquí se usa como punto de partida para asegurar la coherencia entre ambas entregas.

El documento se organiza en dos partes, de acuerdo con la pauta de la asignatura:

- **Parte 1, Análisis estructurado.** Comprende el *modelo ambiental*, que delimita el sistema frente a su entorno mediante la declaración de propósitos, el diagrama de contexto y la lista de acontecimientos; y el *modelo de comportamiento*, que describe su funcionamiento interno mediante los diagramas de flujo de datos (niveles 0, 1 y 2), el diagrama entidad-relación, las mini especificaciones de los procesos elementales y el diccionario de datos.
- **Parte 2, Diseño estructurado.** Traduce el modelo anterior a una arquitectura implementable: el diagrama de estructura, obtenido de los diagramas de flujo de datos mediante el análisis de transacción y de transformación; el modelo relacional, derivado del diagrama entidad-relación; y la especificación de cada módulo.

Siguiendo a Yourdon (1989), el análisis se plantea como un *modelo esencial*: se supone una tecnología perfecta, por lo que no se modelan aspectos como la falta de cobertura en el patio, la compresión de imágenes o el canal por el que viaja un correo. Esos aspectos, centrales en BeeTracer, no se pierden: reaparecen en la Parte 2 como módulos de implementación (sincronización diferida, compresión de fotografías, envío de correo), que es donde corresponde tratarlos. Al igual que en el informe anterior, los procesos, datos y módulos presentados son una propuesta inferida, coherente con el comportamiento observado, pero no necesariamente idéntica a la implementación real de BeeTracer.

2. DESCRIPCIÓN GENERAL

BeeTracer es una plataforma de trazabilidad logística desarrollada por Bilix Ingeniería (Valparaíso) que digitaliza el ciclo de una operación de desconsolidado en terminales extraportuarios y bodegas. El comercio exterior chileno opera mayoritariamente con carga consolidada: la mercancía de varios importadores pequeños viaja en un mismo contenedor, se declara en un manifiesto electrónico que la nave transmite al Servicio Nacional de Aduanas y, como el puerto no tiene espacio para abrir contenedores, se separa por cliente en un puerto seco. Ese momento, el desconsolidado, es donde la custodia de la carga cambia de manos varias veces en pocas horas y donde históricamente la información se registraba de forma más precaria: planillas en papel, fotografías dispersas en teléfonos personales y ninguna prueba del estado de la carga al abrir el contenedor.

La propuesta central de BeeTracer es la **evidencia fotográfica obligatoria**: la aplicación no permite avanzar de una etapa del desconsolidado a la siguiente sin capturar el registro visual correspondiente, de modo que al cerrar la faena existe un expediente completo y verificable. El sistema tiene una arquitectura cliente-servidor con dos componentes:

- **Plataforma web**, usada desde la oficina para configurar los datos maestros, importar manifiestos, programar faenas, validar reportes y consultar el estado de las operaciones.
- **Aplicación móvil**, usada en el patio por el jefe tarjador para registrar el sello, los lotes separados por cliente, las incidencias y las fotografías. Funciona sin conexión y sincroniza la información cuando recupera señal.

Desde la perspectiva estructurada, el sistema se entiende como una red de procesos que transforman flujos de datos. El ciclo completo se descompone en seis procesos principales, que serán la base de los diagramas de flujo de datos y, más adelante, de los subsistemas del diagrama de estructura:

1. **Administrar datos maestros**: el Administrador del depósito mantiene los usuarios, roles y permisos, tipos de bulto, clientes, trabajadores y cuadrillas.
2. **Importar manifiesto**: el Jefe de faena solicita el manifiesto XML de una nave y viaje al Servicio Nacional de Aduanas; el sistema lo valida y registra sus contenedores y líneas de carga antes del arribo.
3. **Programar faenas**: el Jefe de faena planifica el desconsolidado de cada contenedor (fecha, franja horaria, cuadrilla y jefe tarjador), y la faena queda disponible en la aplicación móvil.
4. **Ejecutar operación en terreno**: el Jefe tarjador verifica el sello, registra cada lote separado por cliente, las incidencias (daño, faltante, sobrante, mal embalaje, sello alterado) y cierra la faena con la fotografía del contenedor vacío. Si no hay manifiesto, la carga se ingresa en modalidad *tarja ciega*.
5. **Validar reporte**: al cerrar la faena se genera el borrador del reporte; el Supervisor del

reporte lo revisa y lo aprueba o rechaza la evidencia defectuosa, que se recaptura en terreno.

6. **Distribuir y consultar reportes**: el reporte aprobado se envía al cliente; el Jefe de faena consulta las operaciones y puede reenviar el reporte, y el Cliente del depósito consulta el estado de sus operaciones y descarga sus reportes aprobados.

Respecto del Informe 2 se mantienen los mismos actores, reglas de negocio y estados. Solo se incorporan tres precisiones que el enfoque de datos hace visibles: cada fotografía registra la *etapa* del desconsolidado a la que pertenece (sello, apertura, lote, incidencia o cierre), lo que permite ordenar el reporte por etapa y cliente; cada envío del reporte se registra como un hecho independiente (*envío de reporte*), lo que permite auditar los reenvíos; y, cuando no existe manifiesto, el Jefe de faena puede registrar manualmente el contenedor al programarlo, condición necesaria para operar en tarja ciega.

3. MODELO AMBIENTAL

El modelo ambiental define la frontera entre el sistema y su entorno: qué hace el sistema, con quién se comunica y a qué estímulos debe responder (Yourdon, 1989). Se compone de la declaración de propósitos, el diagrama de contexto y la lista de acontecimientos.

3.1. Declaración de propósitos

El propósito de BeeTracer es **registrar de forma verificable el estado de la carga en cada etapa crítica del desconsolidado de contenedores**, desde la llegada del manifiesto electrónico hasta la entrega del reporte al cliente, para que el terminal extraportuario pueda demostrar, ante un reclamo o un seguro, si la mercancía venía dañada desde origen o si el daño ocurrió durante sus propias maniobras.

Para cumplir ese propósito, el sistema:

1. Conoce el contenido de cada contenedor antes de su arribo, importando el manifiesto electrónico del Servicio Nacional de Aduanas, y admite el ingreso manual de la carga (tarja ciega) cuando el manifiesto no está disponible.
2. Planifica cada faena de desconsolidado, asignando fecha, franja horaria, cuadrilla y jefe tarjador sin conflictos de agenda.
3. Exige evidencia fotográfica obligatoria, fechada y geolocalizada, del sello, la apertura, cada lote separado por cliente, cada incidencia y el contenedor vacío.
4. Contrasta lo recibido con lo declarado y registra las incidencias (daño, faltante, sobrante, mal embalaje y sello alterado) asociadas al cliente afectado.
5. Somete el reporte de cada operación a la validación de un supervisor antes de que llegue al cliente, permitiendo rechazar y recapturar la evidencia defectuosa.
6. Entrega al cliente el reporte aprobado y le permite consultar el estado de sus operaciones, con acceso restringido a la carga consignada a su empresa.

Quedan fuera del propósito del sistema la manipulación física de la carga, el cobro de los servicios del depósito, la gestión del patio de contenedores (visor 3D) y los avisos de transporte, que Bilix comercializa como módulos separados.

3.2. Diagrama de contexto

El diagrama de contexto (Figura 1) representa a BeeTracer como un único proceso, numerado 0, rodeado por los *terminadores*: las personas y sistemas externos que envían o reciben datos. Se identifican seis terminadores, que corresponden a los actores del Informe 2:

- **Administrador del depósito**: envía las solicitudes de mantención de los datos maestros y recibe su resultado.
- **Jefe de faena**: solicita la importación de manifiestos, programa las faenas, consulta las operaciones y solicita el reenvío de reportes.
- **Servicio Nacional de Aduanas**: sistema externo que recibe la solicitud de manifiesto, entrega el manifiesto XML y recibe la notificación del resultado de la importación.
- **Jefe tarjador**: se identifica en la aplicación móvil, recibe sus faenas asignadas, envía el registro de terreno (datos y fotografías) y, si se le solicita, recaptura la evidencia rechazada.
- **Supervisor del reporte**: solicita la revisión de los reportes, recibe su contenido, decide su aprobación o rechazo y recibe el resultado del envío.
- **Cliente del depósito**: consulta el estado de sus operaciones y recibe el reporte PDF aprobado.

La cuadrilla no aparece como terminador porque no intercambia datos con el sistema: su trabajo se registra a través del jefe tarjador. Tampoco aparece el servidor de correo, ya que es el *medio* por el cual el reporte llega al cliente y no un origen o destino de datos; en el modelo esencial el flujo *reporte PDF* va directamente al Cliente del depósito.

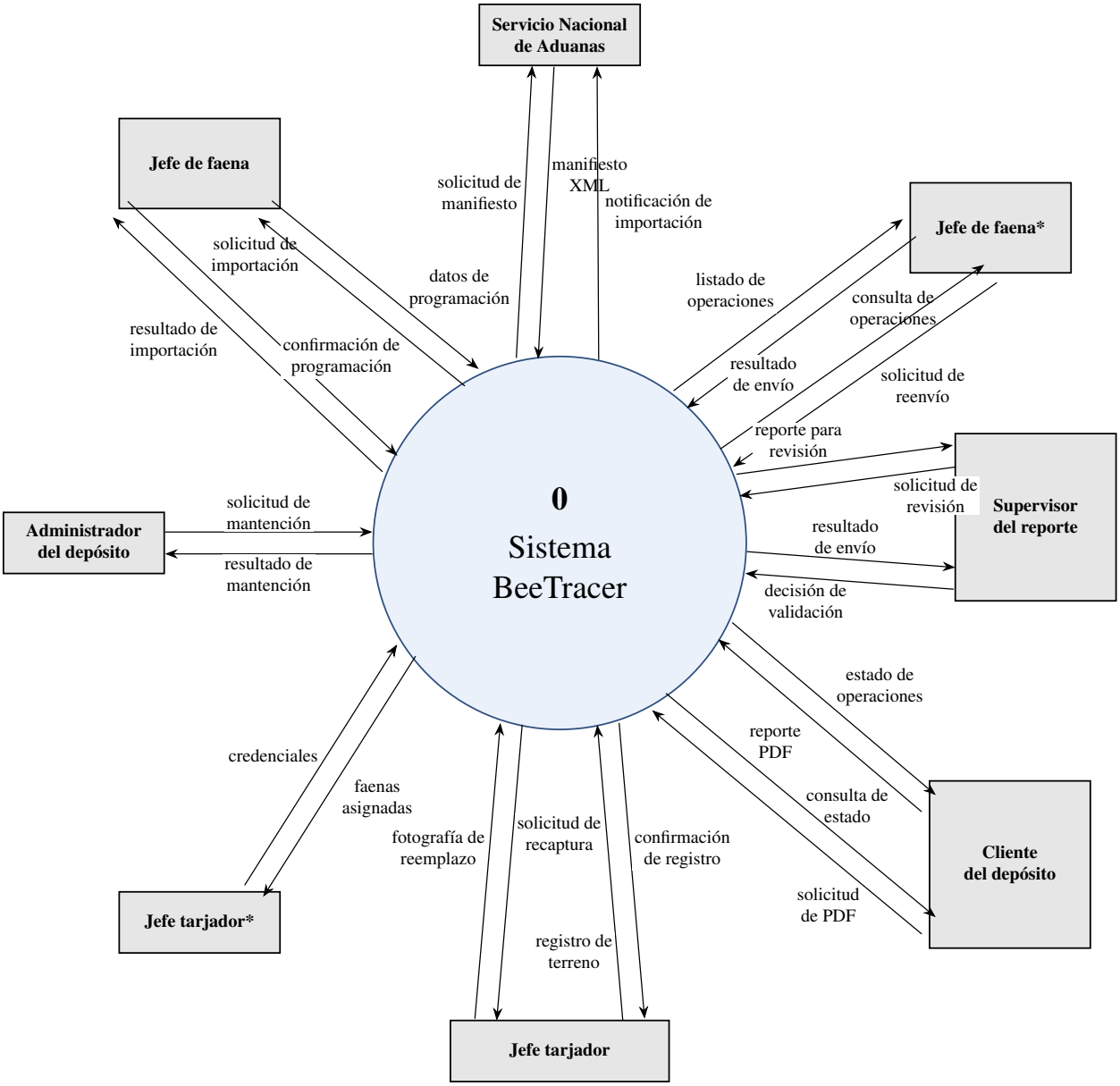


Figura 1: Diagrama de contexto del sistema BeeTracer.

3.3. Lista de acontecimientos

La lista de acontecimientos enumera los sucesos del entorno ante los cuales el sistema debe responder. Cada acontecimiento se clasifica según Yourdon (1989) como de **flujo (F)**, cuando llega un flujo de datos al sistema; **temporal (T)**, cuando lo dispara el paso del tiempo; o de **control (C)**, cuando lo dispara una señal sin datos. Todos los acontecimientos de BeeTracer son de flujo: el sistema siempre reacciona a datos que envía un terminador. El único candidato a acontecimiento de control, la recuperación de la señal en el patio, pertenece a la tecnología de implementación y por ello no forma parte del modelo esencial (se trata en el diseño, Capítulo 5).

La Tabla 1 indica, para cada acontecimiento, el flujo de entrada que lo anuncia y el proceso

del diagrama de flujo de datos de nivel 1 que le responde.

Tabla 1: Lista de acontecimientos del sistema BeeTracer.

N.º	Acontecimiento	Tipo	Flujo de entrada	Proceso
1	El Administrador solicita crear, editar, eliminar o cambiar el estado de un usuario.	F	solicitud de mantención (datos de usuario)	1.1
2	El Administrador solicita mantener un rol y los permisos que otorga.	F	solicitud de mantención (datos de rol)	1.2
3	El Administrador solicita mantener un tipo de bulto.	F	solicitud de mantención (datos de tipo de bulto)	1.3
4	El Administrador solicita mantener un cliente y su contacto.	F	solicitud de mantención (datos de cliente)	1.4
5	El Administrador solicita mantener un trabajador, una cuadrilla o la asignación de un trabajador a una cuadrilla.	F	solicitud de mantención (datos de trabajador)	1.5
6	El Jefe de faena solicita importar el manifiesto de una nave y viaje.	F	solicitud de importación	2.1
7	El Servicio Nacional de Aduanas entrega el manifiesto solicitado.	F	manifiesto XML	2.2
8	El Jefe de faena crea la programación del desconsolidado de un contenedor.	F	datos de programación (nueva programación)	3.1
9	El Jefe de faena asigna la cuadrilla y el jefe tarjador de una programación.	F	datos de programación (asignación)	3.2
10	El Jefe de faena reprograma, anula, elimina o cambia el estado de una programación.	F	datos de programación (cambio de programación)	3.3
11	El Jefe tarjador inicia sesión en la aplicación móvil y solicita sus faenas asignadas.	F	credenciales	3.4
12	El Jefe tarjador inicia la operación de una faena asignada.	F	registro de terreno (datos de inicio)	4.1
13	El Jefe tarjador registra el sello del contenedor y su fotografía.	F	registro de terreno (datos de sello)	4.2
14	El Jefe tarjador registra un lote separado por cliente y su fotografía.	F	registro de terreno (datos de lote)	4.3
15	El Jefe tarjador registra una incidencia con sus fotografías de respaldo.	F	registro de terreno (datos de incidencia)	4.4
16	El Jefe tarjador cierra la faena con la fotografía del contenedor vacío.	F	registro de terreno (datos de cierre)	4.5
17	El Supervisor del reporte solicita revisar un reporte pendiente.	F	solicitud de revisión	5.2
18	El Supervisor del reporte aprueba un reporte.	F	decisión de validación (aprobación)	5.3
19	El Supervisor del reporte rechaza una fotografía defectuosa y registra una observación.	F	decisión de validación (rechazo)	5.4

N.º	Acontecimiento	Tipo	Flujo de entrada	Proceso
20	El Jefe tarjador envía la fotografía que reemplaza a la rechazada.	F	fotografía de reemplazo	5.5
21	El Jefe de faena consulta las operaciones o modifica el estado de una de ellas.	F	consulta de operaciones	6.2
22	El Jefe de faena solicita reenviar al cliente un reporte aprobado.	F	solicitud de reenvío	6.1
23	El Cliente del depósito consulta el estado de sus operaciones.	F	consulta de estado	6.3
24	El Cliente del depósito solicita visualizar o descargar un reporte aprobado.	F	solicitud de PDF	6.4

Algunas respuestas del sistema no tienen un acontecimiento propio porque son consecuencia de otro: al cerrarse la faena (acontecimiento 16) se genera el borrador del reporte (proceso 5.1) y al aprobarse un reporte (acontecimiento 18) se envía al cliente (proceso 6.1). Estas cadenas se representan en los diagramas de flujo de datos mediante los flujos internos *operación finalizada* y *reporte aprobado*.

4. MODELO DE COMPORTAMIENTO

El modelo de comportamiento describe qué debe hacer el sistema internamente para responder a los acontecimientos del modelo ambiental (Yourdon, 1989). Se compone de los diagramas de flujo de datos, que muestran las funciones y los datos que intercambian; el diagrama entidad-relación, que muestra la estructura de los datos almacenados; las mini especificaciones, que describen la lógica de cada proceso elemental; y el diccionario de datos, que define con precisión cada flujo, almacén y dato elemental.

4.1. Diagramas de flujo de datos

Los diagramas de flujo de datos (DFD) se construyeron con la notación de Yourdon (1989) y DeMarco (1979):

- Los **procesos** se dibujan como círculos, con un número jerárquico y un nombre de la forma verbo más objeto. El proceso 4.3 del nivel 1 se descompone en los procesos 4.3.1, 4.3.2 y 4.3.3 del nivel 2, y así sucesivamente.
- Los **terminadores** se dibujan como rectángulos y los **almacenes** como dos líneas paralelas, identificados con la letra D y un número (D1 a D9).
- Los **flujos** son flechas rotuladas con el nombre del dato que transportan. Los flujos hacia o desde un almacén no se rotulan cuando transportan registros completos del almacén; la flecha de doble punta indica que el proceso lee y actualiza el almacén.
- Para evitar cruces, algunos terminadores y almacenes se dibujan más de una vez; las copias se marcan con asterisco (por ejemplo, *Jefe de faena** o *D2* Clientes*).
- En los diagramas hijos, los flujos que vienen de otro proceso del diagrama padre o van hacia él se indican con el texto *desde el proceso n* o *hacia el proceso n*.

Cada diagrama hijo está **balanceado** con su padre: los flujos que entran y salen de un proceso en el nivel superior son los mismos que entran y salen de su diagrama hijo, ya sea directamente o mediante sus componentes. Por ejemplo, el flujo *solicitud de mantención* del nivel 0 se descompone en el nivel 1 en *datos de usuario*, *datos de rol*, *datos de tipo de bulto*, *datos de cliente* y *datos de trabajador*, tal como lo define el diccionario de datos (Sección 4.4).

4.1.1. DFD de nivel 0

El DFD de nivel 0 (Figura 2) descompone el proceso único del diagrama de contexto en los seis procesos principales del sistema y muestra los nueve almacenes de datos que comparten. Se lee de izquierda a derecha siguiendo el ciclo de vida de una operación:

1. **Administrar datos maestros** mantiene los almacenes de usuarios (D1), clientes (D2), trabajadores y cuadrillas (D3) y tipos de bulto (D4), que los demás procesos solo consultan.

2. **Importar manifiesto** obtiene el manifiesto XML del Servicio Nacional de Aduanas y registra sus contenedores y líneas de carga en D5.
3. **Programar faenas** crea la programación de cada contenedor en D6, asigna la cuadrilla y el jefe tarjador y entrega a este último sus faenas asignadas.
4. **Ejecutar operación en terreno** registra el avance de la faena en D7 y la evidencia fotográfica en D8; al cerrar la faena emite el flujo interno *operación finalizada*.
5. **Validar reporte** genera el borrador del reporte en D9, lo presenta al Supervisor y lo aprueba o rechaza; al aprobarlo emite el flujo interno *reporte aprobado*.
6. **Distribuir y consultar reportes** envía el reporte aprobado al cliente, permite su reenvío y atiende las consultas del Jefe de faena y del Cliente del depósito.

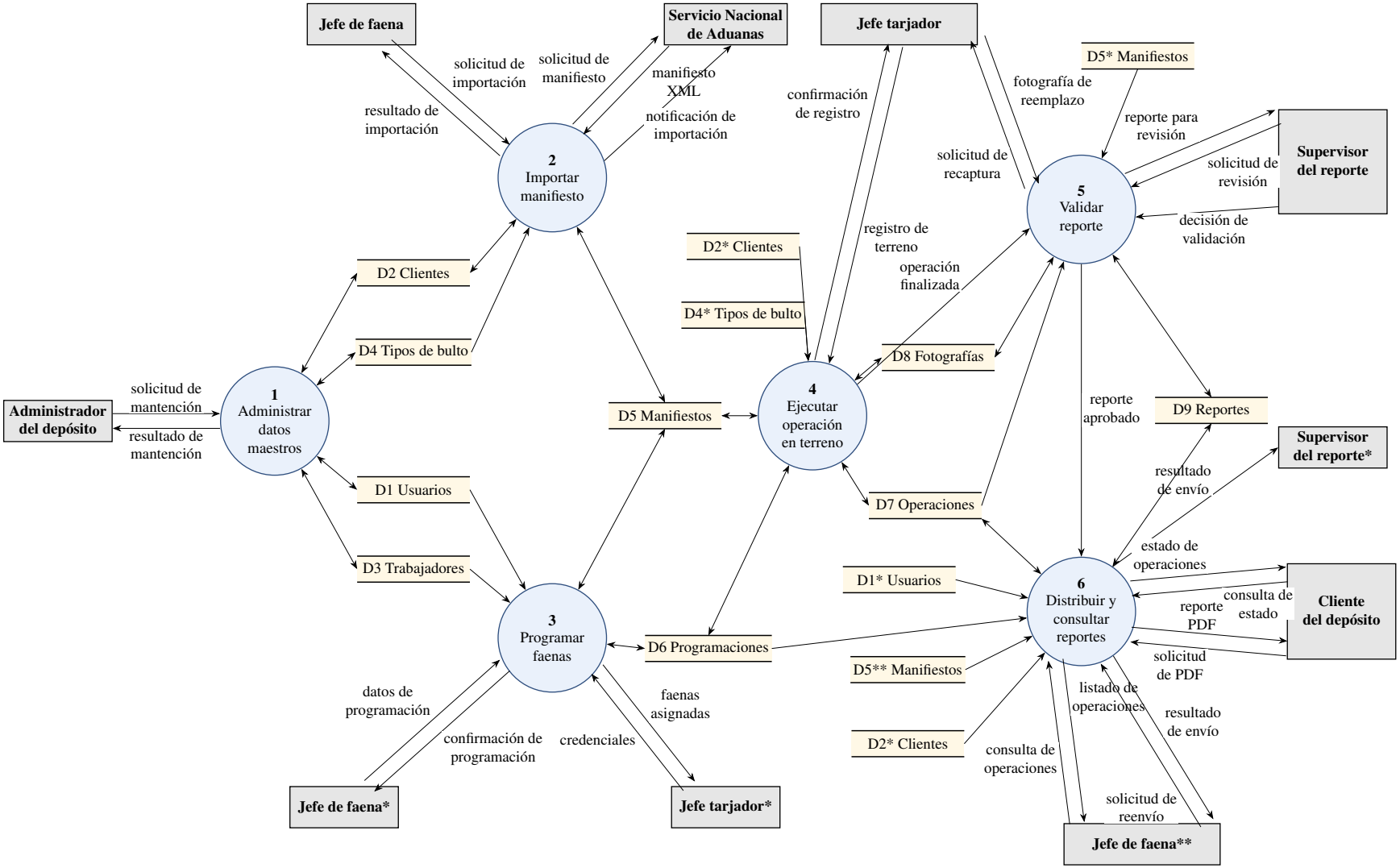


Figura 2: Diagrama de flujo de datos de nivel 0 del sistema BeeTracer.

4.1.2. DFD de nivel 1

En el nivel 1 se descompone cada proceso del nivel 0 en los subprocesos que lo componen. A continuación se describe cada diagrama; los seis diagramas se presentan juntos al final de esta sección (Figuras 3 a 8).

Proceso 1: Administrar datos maestros. La solicitud de mantención del Administrador se separa según el tipo de dato maestro (Figura 3). Cada subproceso valida los datos, aplica la acción solicitada (crear, editar, eliminar o cambiar el estado) sobre su almacén y devuelve el resultado. El proceso 1.1 consulta además los almacenes de clientes y trabajadores, porque una cuenta de usuario puede quedar ligada a la ficha de un trabajador (jefe tarjador) o de un cliente.

Proceso 2: Importar manifiesto. El proceso 2.1 verifica que el viaje no haya sido importado antes y solicita el manifiesto a Aduanas; el 2.2 valida la estructura y los datos del XML recibido; si es válido, el 2.3 lo registra y, si no, el 2.4 genera el reporte de errores sin registrar información parcial. En ambos casos el 2.5 notifica el resultado a Aduanas y entrega el resumen al Jefe de faena (Figura 4).

Proceso 3: Programar faenas. El Jefe de faena crea la programación de un contenedor pendiente (3.1), le asigna la cuadrilla y el jefe tarjador (3.2) y puede reprogramarla, anularla, eliminarla o cambiar su estado (3.3). Cuando el jefe tarjador inicia sesión en la aplicación móvil, el proceso 3.4 le entrega las faenas que tiene asignadas, con los datos del contenedor y su carga declarada (Figura 5).

Proceso 4: Ejecutar operación en terreno. Cada etapa del flujo guiado del jefe tarjador corresponde a un subproceso (Figura 6): iniciar la operación (4.1), verificar el sello (4.2), registrar cada lote separado por cliente (4.3), registrar las incidencias (4.4) y cerrar la faena (4.5). Los procesos que reciben una fotografía la registran en el almacén D8 junto con su etapa, fecha, hora y geoposición, lo que materializa la regla de evidencia obligatoria. Al cerrar la faena, el proceso 4.5 emite el flujo *operación finalizada* hacia el proceso 5.

Proceso 5: Validar reporte. El proceso 5.1 genera el borrador del reporte cuando recibe una operación finalizada o una evidencia corregida. El Supervisor solicita revisar un reporte (5.2) y lo aprueba (5.3) o rechaza la fotografía defectuosa (5.4), lo que genera una solicitud de recaptura al jefe tarjador. La fotografía de reemplazo se registra en el proceso 5.5, que reactiva la generación de una nueva versión del reporte (Figura 7).

Proceso 6: Distribuir y consultar reportes. El proceso 6.1 envía el reporte aprobado a los correos del cliente, ya sea automáticamente al aprobarse o a solicitud del Jefe de faena, y registra cada envío. El Jefe de faena consulta las operaciones y modifica su estado (6.2), y el Cliente del

depósito consulta el estado de sus operaciones (6.3) y descarga el reporte aprobado (6.4), viendo solo las operaciones con carga consignada a su empresa (Figura 8).

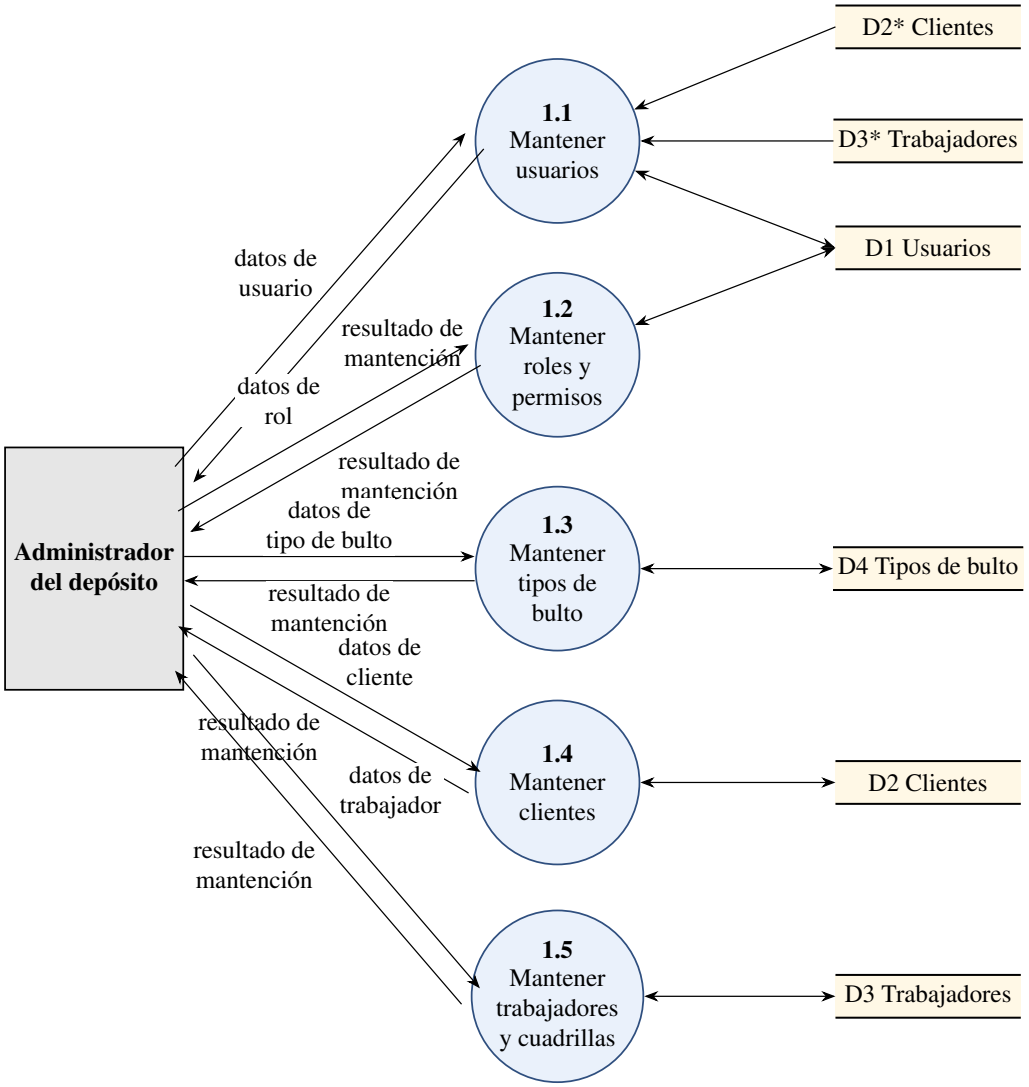


Figura 3: DFD de nivel 1 - Proceso 1: Administrar datos maestros.

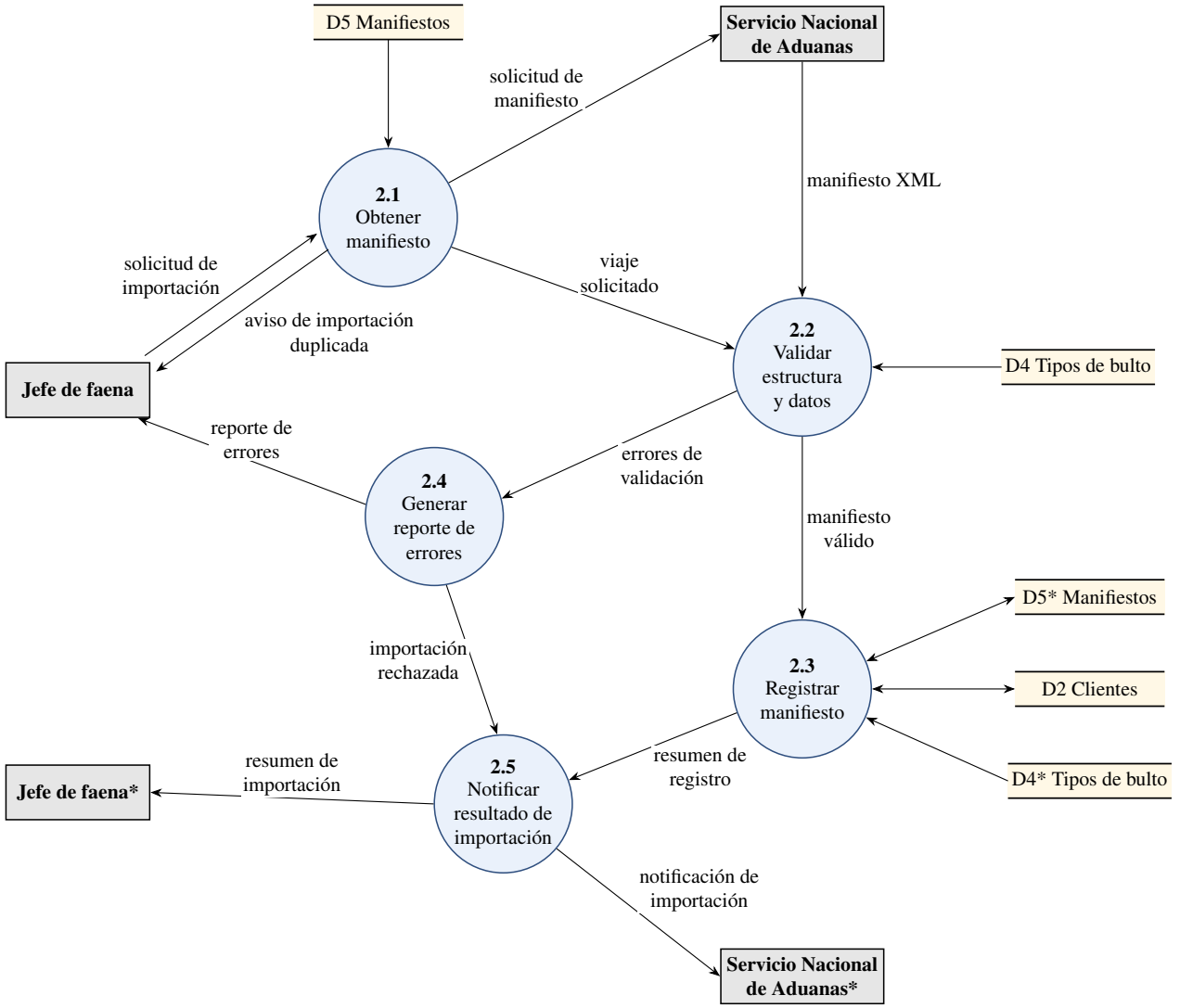


Figura 4: DFD de nivel 1 - Proceso 2: Importar manifiesto.

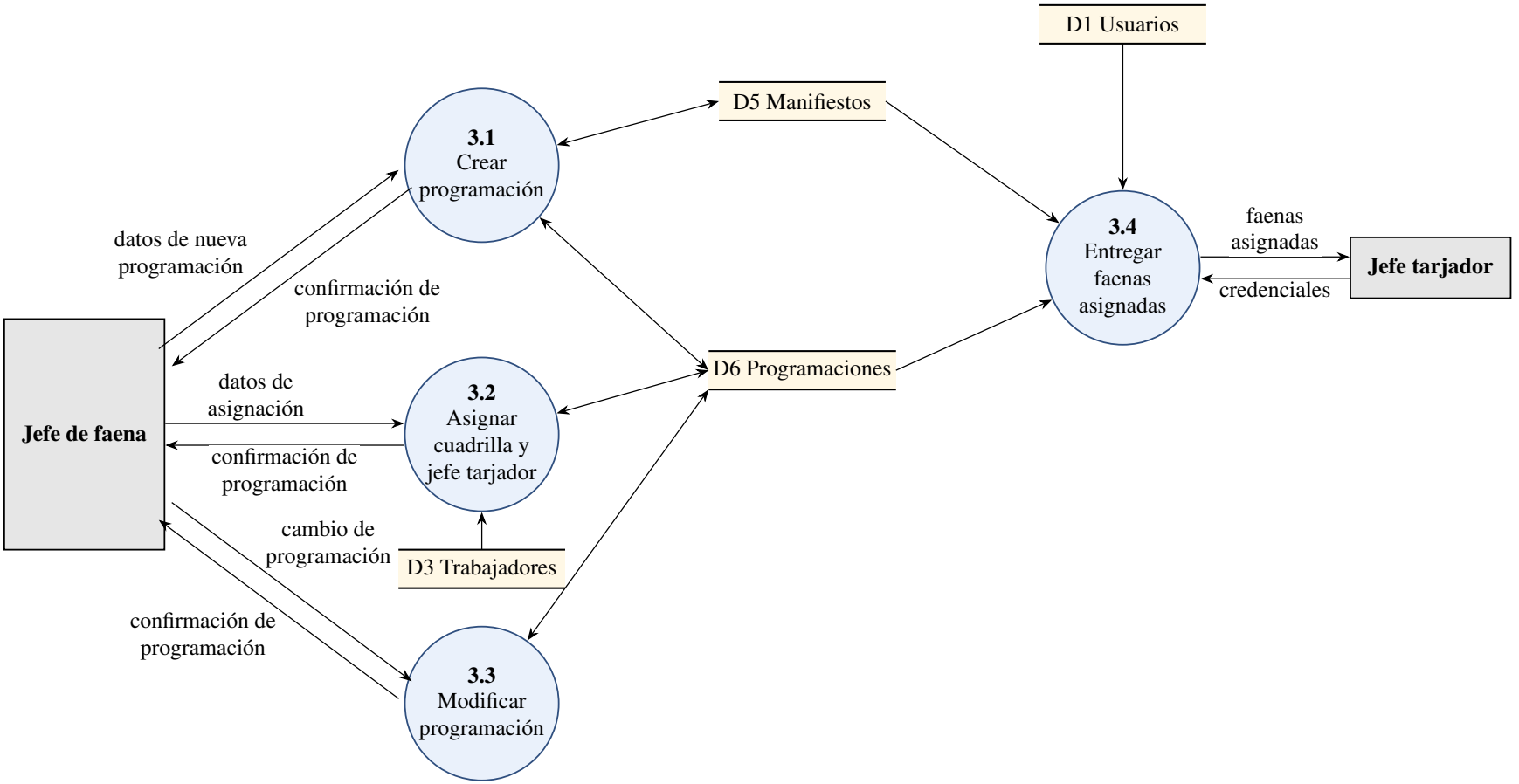


Figura 5: DFD de nivel 1 - Proceso 3: Programar faenas.

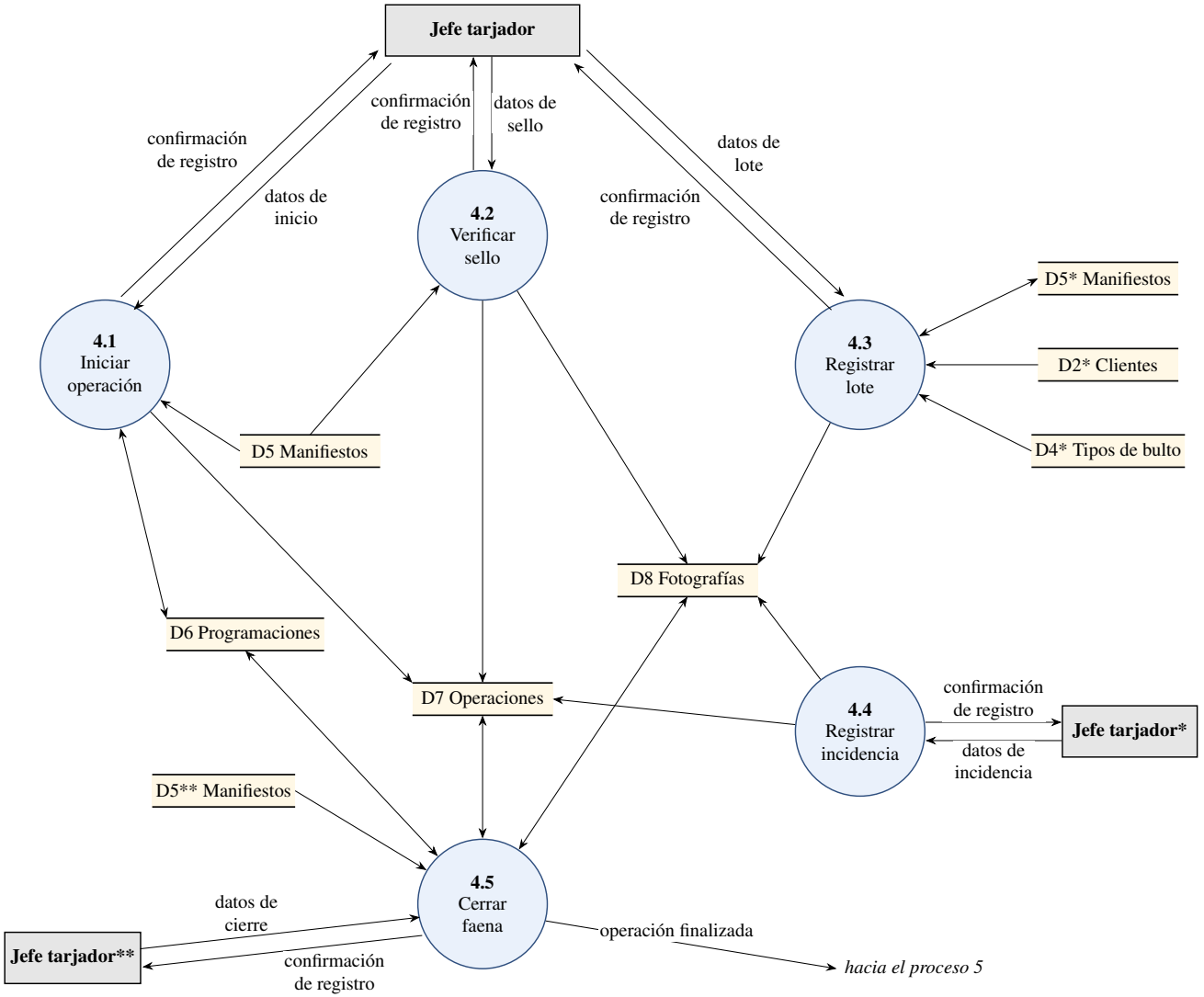


Figura 6: DFD de nivel 1 - Proceso 4: Ejecutar operación en terreno.

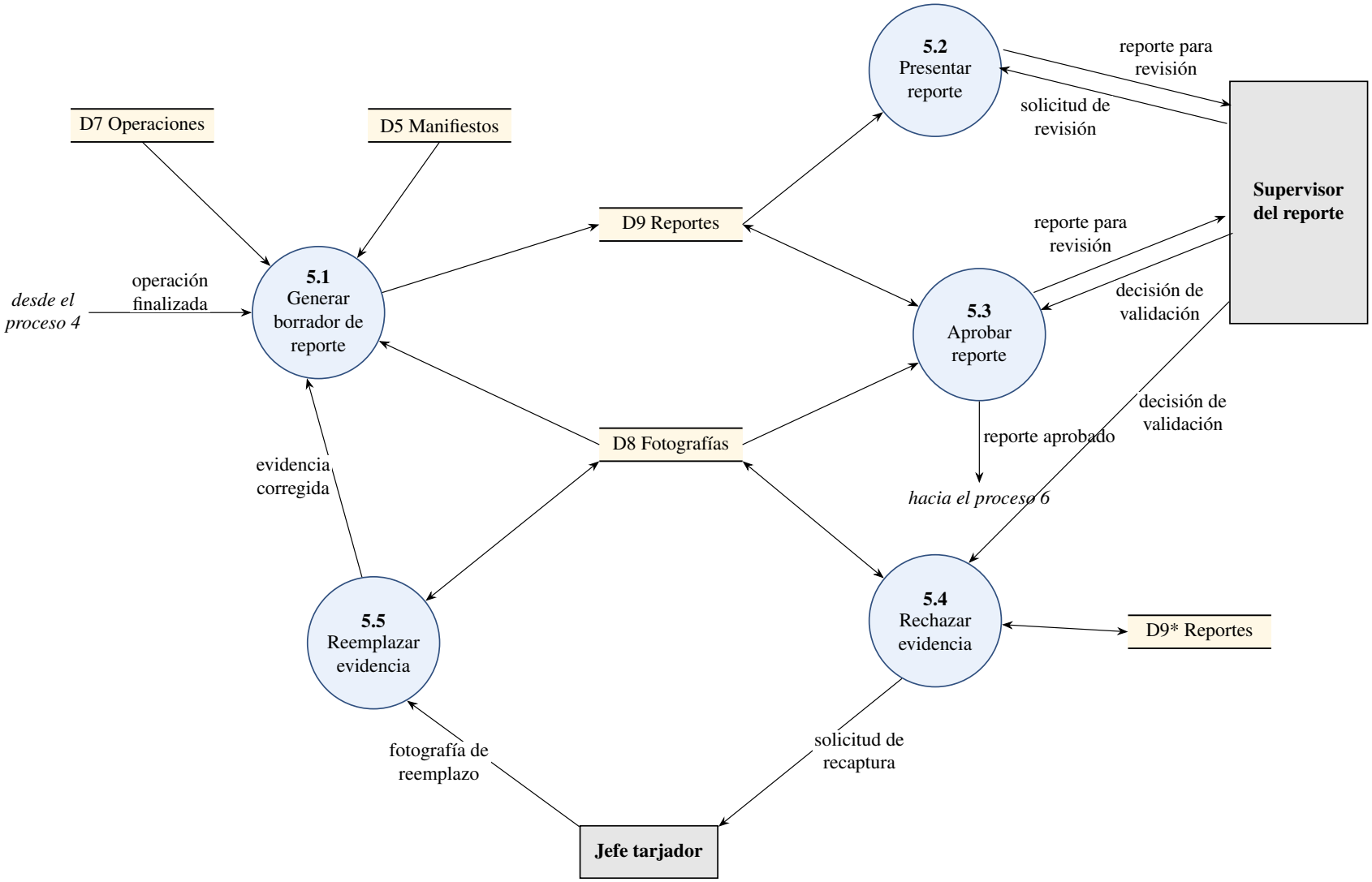


Figura 7: DFD de nivel 1 - Proceso 5: Validar reporte.

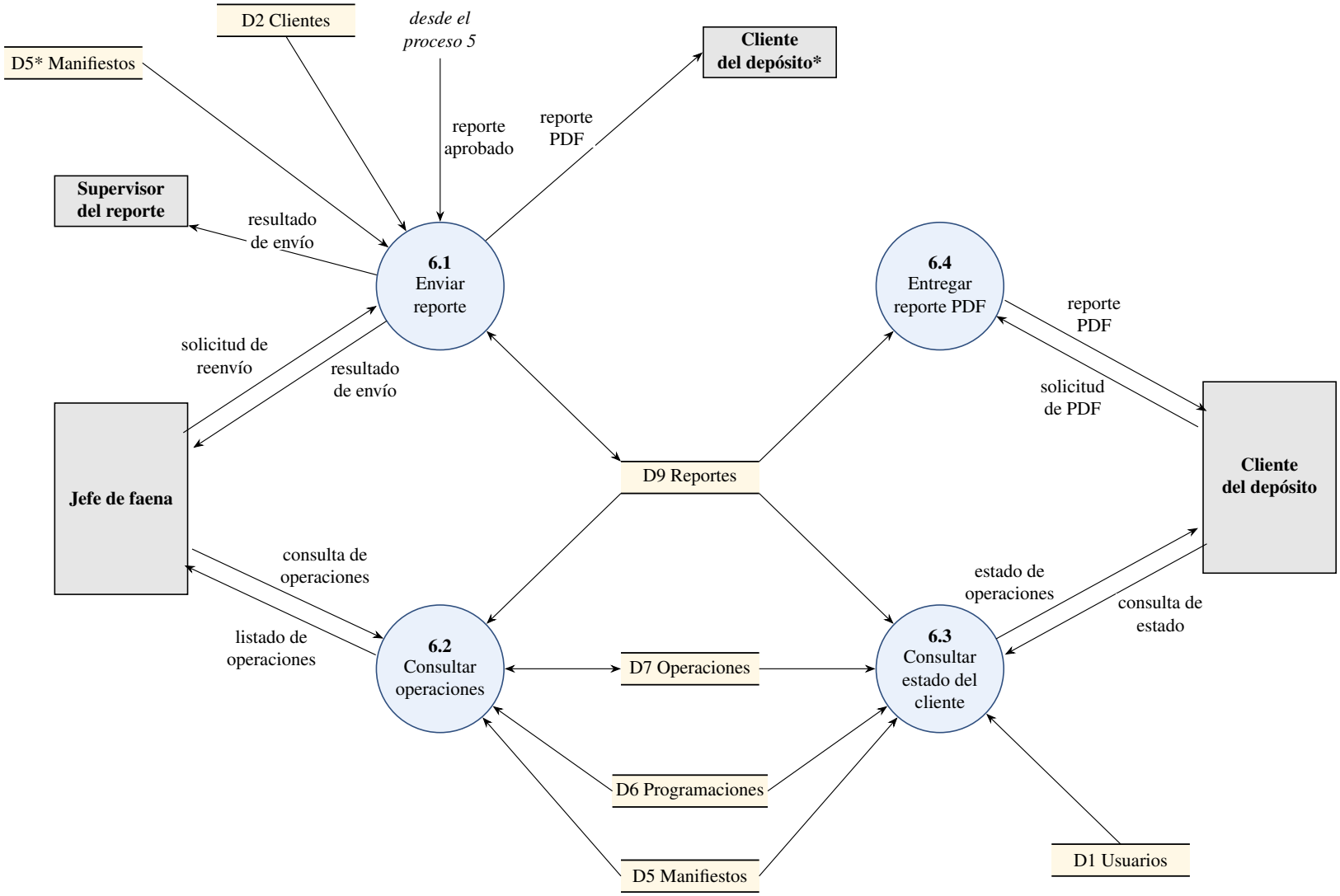


Figura 8: DFD de nivel 1 - Proceso 6: Distribuir y consultar reportes.

4.1.3. DFD de nivel 2

Se descomponen en un nivel más los tres procesos de nivel 1 cuya lógica involucra varios pasos con almacenes distintos. Los demás procesos de nivel 1 se consideran *elementales*: su lógica cabe en una mini especificación de una página.

Proceso 2.3: Registrar manifiesto. El registro del manifiesto válido se realiza en cadena (Figura 9): primero la cabecera (nave, viaje y número), luego cada contenedor declarado y, por cada línea de carga, se verifica que el consignatario exista en el almacén de clientes, creándolo con sus datos mínimos (RUT y razón social) si no existe, antes de registrar la línea con su tipo de bulto.

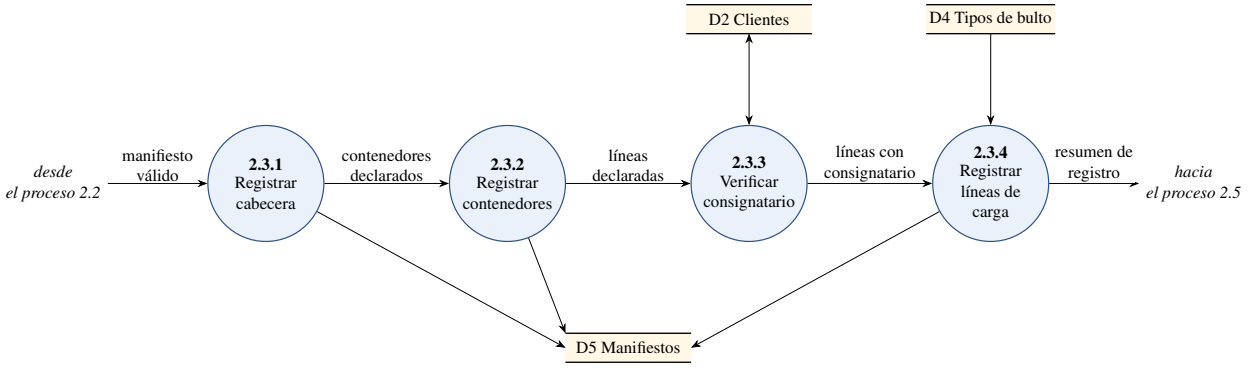


Figura 9: DFD de nivel 2 - Proceso 2.3: Registrar manifiesto.

Proceso 3.2: Asignar cuadrilla y jefe tarjador. La asignación es un diálogo con el Jefe de faena (Figura 10): el sistema determina qué cuadrillas y jefes tarjadores están habilitados y libres en la franja de la programación (3.2.1); el Jefe de faena elige y el sistema vuelve a verificar la disponibilidad, porque otra programación pudo ocuparlos entretanto (3.2.2); finalmente registra la asignación (3.2.3).

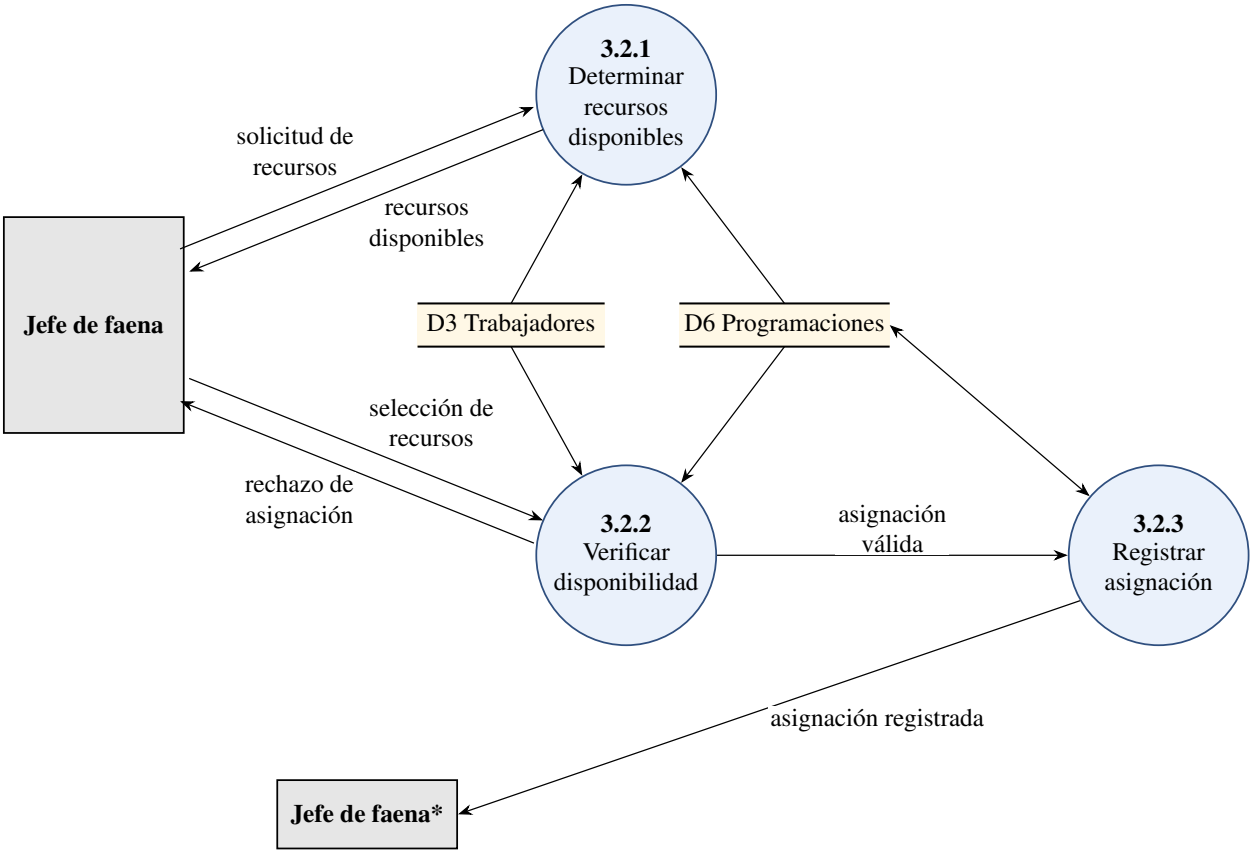


Figura 10: DFD de nivel 2 - Proceso 3.2: Asignar cuadrilla y jefe tarjador.

Proceso 4.3: Registrar lote. Por cada lote separado (Figura 11), el sistema identifica la línea de carga a la que pertenece o, en tarja ciega, la crea con los datos ingresados (4.3.1); contrasta la cantidad recibida con la declarada y registra la fotografía del lote (4.3.2); y evalúa la diferencia para confirmar el lote o exigir el registro de una incidencia de faltante, sobrante o daño (4.3.3).

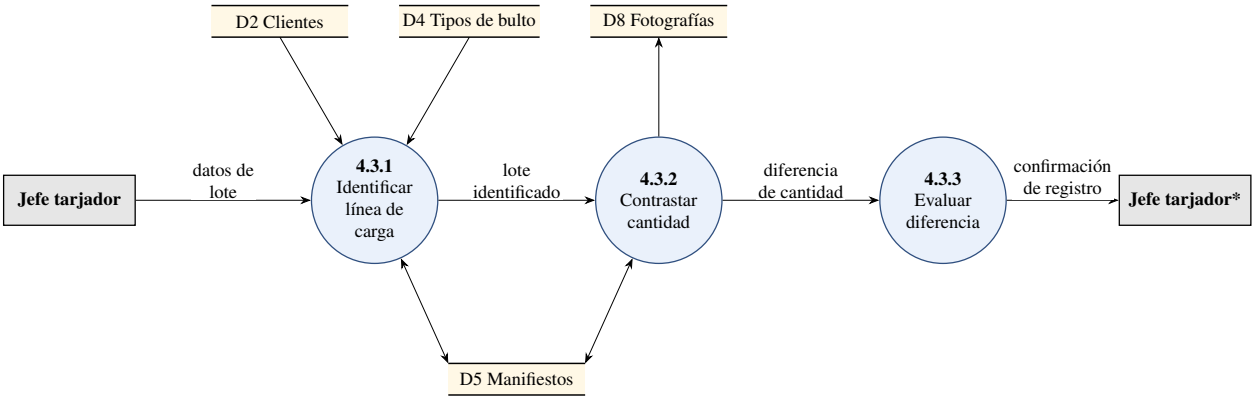


Figura 11: DFD de nivel 2 - Proceso 4.3: Registrar lote.

4.2. Diagrama entidad-relación

El diagrama entidad-relación (DER) describe la estructura lógica de los datos que el sistema almacena, con independencia de los procesos que los usan (Chen, 1976). Se construyó a partir del contenido de los almacenes D1 a D9 y es coherente con las clases de entidad del diagrama de clases del Informe 2: cada clase persistente se convierte en una entidad y las clases controladoras y de servicio (`GestorManifiesto`, `ServicioCorreo`, `SincronizadorOffline`, etc.) no aparecen, porque no representan datos sino comportamiento.

Se usa la notación de Chen: las entidades son rectángulos, las relaciones son rombos rotulados con un verbo y cada línea indica la cardinalidad máxima de la entidad que la toca (1, N o M). Para no recargar el diagrama, los atributos se listan dentro de cada entidad, con el identificador subrayado (Figura 12). Las claves foráneas no se muestran, ya que son consecuencia de las relaciones y aparecen recién en el modelo relacional (Capítulo 6).

Las relaciones más relevantes del modelo son las siguientes:

- **Seguridad.** Un ROL *otorga* muchos PERMISOs y un permiso puede pertenecer a muchos roles (N:M). Cada USUARIO tiene un único rol y, según su perfil, *corresponde* a la ficha de un TRABAJADOR (el jefe tarjador) o *representa* a un CLIENTE.
- **Carga declarada.** Un MANIFIESTO *declara* muchos CONTENEDORes y cada contenedor *contiene* muchas LINEA_CARGA; cada línea es *consignada* a un cliente y *clasificada* con un TIPO_BULTO. En tarja ciega, el contenedor existe sin manifiesto y sus líneas se crean en terreno.
- **Planificación.** Cada contenedor *se planifica* en una sola PROGRAMACION, que *ejecuta* una CUADRILLA y *dirige* un trabajador en el rol de jefe tarjador. Una cuadrilla *integra* a muchos trabajadores.
- **Operación en terreno.** Una programación *origina* una única OPERACION, que *verifica* un SELLO_SEGURIDAD, *registra* incidencias y *documenta* fotografías. Cada INCIDENCIA *afecta* a una línea de carga (salvo la de sello alterado) y es *respaldada* por fotografías; cada FOTOGRAFIA de lote *evidencia* a una línea de carga.
- **Reporte.** Una operación *genera* un REPORTE, que se versiona con cada corrección, es *aprobado* por un usuario supervisor y *registra* cada ENVIO_REPORTE, automático o reenviado.

Las participaciones opcionales del modelo son: la cuenta de usuario ligada a un trabajador o a un cliente; la cuadrilla y el jefe tarjador de una programación aún no asignada; el manifiesto de un contenedor ingresado en tarja ciega; la línea de carga de una incidencia de sello alterado y de las fotografías que no son de lote; la incidencia de las fotografías que no la respaldan; y el aprobador de un reporte en borrador.

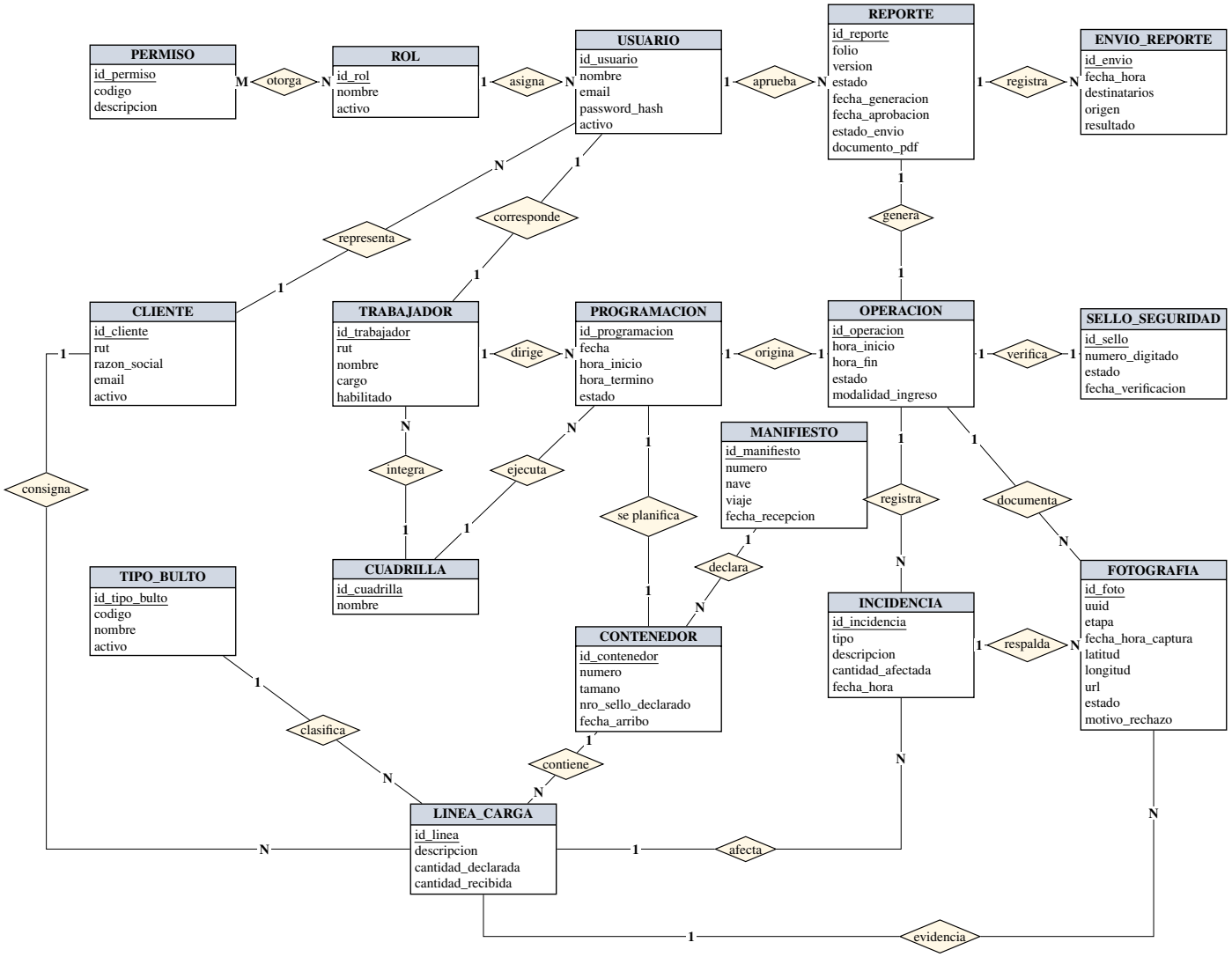


Figura 12: Diagrama entidad-relación del sistema BeeTracer (notación de Chen).

4.3. Mini especificaciones de los procesos elementales

Las mini especificaciones describen la lógica de cada proceso elemental, es decir, de cada proceso que no se descompone en un diagrama de nivel inferior (DeMarco, 1979). Se redactan en **español estructurado**: oraciones imperativas que usan solo los flujos y datos definidos en el diccionario de datos (en cursiva los flujos), combinadas con las estructuras **SI** … **ENTONCES** … **SINO**, **PARA CADA** y **MIENTRAS**. Los verbos **LEER**, **ESCRIBIR** y **ACTUALIZAR** indican el acceso a un almacén, y **ENVIAR** la emisión de un flujo. Cada especificación indica además los flujos de entrada y salida y los almacenes que usa, de modo que pueda contrastarse con su DFD.

4.3.1. Proceso 1: Administrar datos maestros

Los cinco procesos de mantención comparten la misma estructura: validan la solicitud, aplican la acción y devuelven el resultado. La eliminación es *lógica* cuando el registro ya fue usado por otra entidad, para no perder la trazabilidad histórica.

Proceso 1.1: Mantener usuarios

Entradas: *datos de usuario* (desde Administrador del depósito).

Salidas: *resultado de mantención* (hacia Administrador del depósito).

Almacenes: D1 Usuarios (lectura y escritura); D2 Clientes y D3 Trabajadores (lectura).

Lógica:

1. **SI** acción = crear **ENTONCES**
   a. **SI** falta nombre, email, password o id_rol **ENTONCES ENVIAR** *resultado de mantención* con los campos faltantes y terminar.
   b. **LEER** D1; **SI** el email ya existe **ENTONCES ENVIAR** “correo ya registrado” y terminar.
   c. **SI** la password no cumple la política mínima **ENTONCES ENVIAR** sus requisitos y terminar.
   d. **SI** el rol es de jefe tarjador o de cliente **ENTONCES LEER** D3 o D2 y verificar que exista la ficha a la que se ligará la cuenta.
   e. Calcular password_hash; **ESCRIBIR** el usuario en D1 con activo = verdadero.
2. **SI** acción = editar **ENTONCES** validar los campos modificados con las mismas reglas y **ACTUALIZAR** D1.
3. **SI** acción = eliminar o desactivar **ENTONCES ACTUALIZAR** activo = falso en D1.
4. **ENVIAR** *resultado de mantención* con el usuario resultante.

Proceso 1.2: Mantener roles y permisos

Entradas: *datos de rol* (desde Administrador del depósito).

Salidas: *resultado de mantención*.

Almacenes: D1 Usuarios (lectura y escritura de roles y permisos).

Lógica:

1. **SI** acción = crear o editar **ENTONCES**
   a. **SI** el nombre está vacío o ya existe otro rol con ese nombre **ENTONCES ENVIAR** el error y

terminar.
   b. **ESCRIBIR** el rol en D1 y, **PARA CADA** código de permiso seleccionado, asociarlo al rol.
2. **SI** acción = eliminar **ENTONCES**
   a. **LEER** D1; **SI** existen usuarios activos con ese rol **ENTONCES ENVIAR** "rol en uso" y terminar **SINO** eliminar el rol y sus asociaciones.
3. **SI** acción = activar o desactivar **ENTONCES ACTUALIZAR** activo en D1.
4. **ENVIAR** *resultado de mantención*.

Proceso 1.3: Mantener tipos de bulto

Entradas: *datos de tipo de bulto*.

Salidas: *resultado de mantención*.

Almacenes: D4 Tipos de bulto (lectura y escritura).

Lógica:

1. **SI** acción = crear o editar **ENTONCES**
   a. **SI** falta nombre o código, o el código ya existe en D4 **ENTONCES ENVIAR** el error y terminar.
   b. **ESCRIBIR** el tipo de bulto en D4 con activo = verdadero.
2. **SI** acción = eliminar **ENTONCES SI** existen líneas de carga con ese tipo **ENTONCES ACTUALIZAR** activo = falso **SINO** eliminarlo de D4.
3. **SI** acción = activar o desactivar **ENTONCES ACTUALIZAR** activo en D4.
4. **ENVIAR** *resultado de mantención*.

Proceso 1.4: Mantener clientes

Entradas: *datos de cliente*.

Salidas: *resultado de mantención*.

Almacenes: D2 Clientes (lectura y escritura).

Lógica:

1. **SI** acción = crear o editar **ENTONCES**
   a. **SI** el rut no tiene formato válido o su dígito verificador no coincide **ENTONCES ENVIAR** el error y terminar.
   b. **SI** se crea y el rut ya existe en D2 **ENTONCES ENVIAR** "cliente ya registrado" y terminar.
   c. **SI** email no tiene formato válido **ENTONCES ENVIAR** el error y terminar.
   d. **ESCRIBIR** o **ACTUALIZAR** el cliente en D2 (incluye completar la ficha de un cliente creado con datos mínimos desde un manifiesto).
2. **SI** acción = eliminar o desactivar **ENTONCES ACTUALIZAR** activo = falso en D2.
3. **ENVIAR** *resultado de mantención*.

Proceso 1.5: Mantener trabajadores y cuadrillas

Entradas: *datos de trabajador*.

Salidas: *resultado de mantención*.

Almacenes: D3 Trabajadores (lectura y escritura).

Lógica:

1. **SI** el objeto es una cuadrilla **ENTONCES** crear, renombrar o eliminar la cuadrilla en D3; no se elimina si tiene trabajadores asignados.
2. **SI** el objeto es un trabajador **ENTONCES**
   a. **SI** el rut no es válido o ya existe (al crear) **ENTONCES ENVIAR** el error y terminar.
   b. **SI** se indica id_cuadrilla **ENTONCES LEER** D3 y verificar que la cuadrilla exista y tenga menos de cinco integrantes.
   c. **ESCRIBIR** o **ACTUALIZAR** el trabajador en D3; deshabilitar equivale a **ACTUALIZAR** habilitado = falso.
3. **ENVIAR** *resultado de mantención*.

4.3.2. Proceso 2: Importar manifiesto

Proceso 2.1: Obtener manifiesto

Entradas: *solicitud de importación* (desde Jefe de faena).

Salidas: *solicitud de manifiesto* (hacia Servicio Nacional de Aduanas); *viaje solicitado* (hacia 2.2); *aviso de importación duplicada* (hacia Jefe de faena).

Almacenes: D5 Manifiestos (lectura).

Lógica:

1. **SI** falta nave, viaje o fecha **ENTONCES** solicitar los datos y terminar.
2. **LEER** D5 buscando un manifiesto con la misma nave y viaje.
3. **SI** existe y el Jefe de faena no confirma reemplazarlo **ENTONCES ENVIAR** *aviso de importación duplicada* y terminar.
4. **ENVIAR** *solicitud de manifiesto* con nave, viaje y fecha.
5. **ENVIAR** *viaje solicitado* a 2.2, indicando si se trata de un reemplazo.

Proceso 2.2: Validar estructura y datos

Entradas: *manifiesto XML* (desde Servicio Nacional de Aduanas); *viaje solicitado* (desde 2.1).

Salidas: *manifiesto válido* (hacia 2.3); *errores de validación* (hacia 2.4).

Almacenes: D4 Tipos de bulto (lectura).

Lógica:

1. **SI** no se recibe *manifiesto XML* o Aduanas informa que no existe **ENTONCES ENVIAR** *errores de validación* con “manifiesto no disponible” y terminar.
2. Verificar que el XML cumpla el esquema; **PARA CADA** incumplimiento, agregar un error con su ubicación.
3. Verificar que la nave y el viaje del XML coincidan con *viaje solicitado*.
4. **PARA CADA** contenedor: verificar número (cuatro letras y siete dígitos), tamaño y nro_sello_declarado.

5. **PARA CADA** línea: verificar cantidad_declarada > 0, rut del consignatario y que el código de tipo de bulto exista en D4.
6. **SI** la lista de errores está vacía **ENTONCES ENVIAR** *manifiesto válido* **SINO ENVIAR** *errores de validación*.

Proceso 2.3.1: Registrar cabecera

Entradas: *manifiesto válido* (desde 2.2).

Salidas: *contenedores declarados* (hacia 2.3.2).

Almacenes: D5 Manifiestos (escritura).

Lógica:

1. **SI** se trata de un reemplazo **ENTONCES** eliminar de D5 el manifiesto anterior con sus contenedores y líneas que aún no tengan programación.
2. **ESCRIBIR** en D5 el manifiesto con numero, nave, viaje y fecha_recepcion = fecha y hora actuales.
3. **ENVIAR** *contenedores declarados* con el id_manifiesto asignado.

Proceso 2.3.2: Registrar contenedores

Entradas: *contenedores declarados* (desde 2.3.1).

Salidas: *líneas declaradas* (hacia 2.3.3).

Almacenes: D5 Manifiestos (escritura).

Lógica:

1. **PARA CADA** contenedor declarado:
   a. **ESCRIBIR** en D5 el contenedor con numero, tamano, nro_sello_declarado, fecha_arribo e id_manifiesto.
   b. Agregar sus líneas declaradas, con el id_contenedor asignado, a *líneas declaradas*.
2. **ENVIAR** *líneas declaradas* junto con la cantidad de contenedores registrados.

Proceso 2.3.3: Verificar consignatario

Entradas: *líneas declaradas* (desde 2.3.2).

Salidas: *líneas con consignatario* (hacia 2.3.4).

Almacenes: D2 Clientes (lectura y escritura).

Lógica:

1. **PARA CADA** línea declarada:
   a. **LEER** D2 buscando el cliente por el rut del consignatario.
   b. **SI** no existe **ENTONCES ESCRIBIR** en D2 un cliente con rut y razon_social del manifiesto, activo = verdadero, y registrar la advertencia “ficha de cliente incompleta”.
   c. Asociar a la línea el id_cliente encontrado o creado.
2. **ENVIAR** *líneas con consignatario* con las advertencias acumuladas.

Proceso 2.3.4: Registrar líneas de carga

Entradas: *líneas con consignatario* (desde 2.3.3).

Salidas: *resumen de registro* (hacia 2.5).

Almacenes: D4 Tipos de bulto (lectura); D5 Manifiestos (escritura).

Lógica:

1. **PARA CADA** línea: **LEER** en D4 el id_tipo_bulto que corresponde a su código y **ESCRIBIR** en D5 la línea con descripcion, cantidad_declarada, cantidad_recibida = 0, id_contenedor, id_cliente e id_tipo_bulto.
2. **ENVIAR** *resumen de registro* con la cantidad de contenedores, la cantidad de líneas y las advertencias.

Proceso 2.4: Generar reporte de errores

Entradas: *errores de validación* (desde 2.2).

Salidas: *reporte de errores* (hacia Jefe de faena); *importación rechazada* (hacia 2.5).

Almacenes: Ninguno.

Lógica:

1. Ordenar los errores por ubicación en el XML y agruparlos por tipo.
2. Componer el *reporte de errores* con nave, viaje, fecha y la lista de errores.
3. **SI** el error es "manifiesto no disponible" **ENTONCES** agregar la sugerencia de reintentar más tarde o de operar en tarja ciega.
4. **ENVIAR** *reporte de errores* y **ENVIAR** *importación rechazada*.

Proceso 2.5: Notificar resultado de importación

Entradas: *resumen de registro* (desde 2.3.4) o *importación rechazada* (desde 2.4).

Salidas: *notificación de importación* (hacia Servicio Nacional de Aduanas); *resumen de importación* (hacia Jefe de faena).

Almacenes: Ninguno.

Lógica:

1. **SI** se recibe *resumen de registro* **ENTONCES** resultado = exitoso **SINO** resultado = rechazado.
2. **ENVIAR** *notificación de importación* con nave, viaje y resultado.
3. **SI** resultado = exitoso **ENTONCES ENVIAR** *resumen de importación* con la cantidad de contenedores, líneas y advertencias.

4.3.3. Proceso 3: Programar faenas

Proceso 3.1: Crear programación

Entradas: *datos de nueva programación* (desde Jefe de faena).

Salidas: *confirmación de programación* (hacia Jefe de faena).

Almacenes: D5 Manifiestos (lectura y escritura); D6 Programaciones (lectura y escritura).

Lógica:

1. **SI** la solicitud es listar pendientes **ENTONCES LEER** D5 y D6 y **ENVIAR** *confirmación de programación* con los contenedores sin programación, su fecha de arribo y su carga declarada; terminar.
2. **SI** el contenedor no proviene de un manifiesto (contingencia de tarja ciega) **ENTONCES ESCRIBIR** en D5 el contenedor con numero, tamano y nro_sello_declarado.
3. **SI** hora_inicio $\geq$ hora_termino o la fecha es anterior a hoy **ENTONCES ENVIAR** el error y terminar.
4. **LEER** D6; **SI** el contenedor ya tiene programación **ENTONCES ENVIAR** "contenedor ya programado" y terminar.
5. **ESCRIBIR** en D6 la programación con estado = PROGRAMADA, sin cuadrilla ni jefe tarjador.
6. **ENVIAR** *confirmación de programación* con el id_programacion.

Proceso 3.2.1: Determinar recursos disponibles

Entradas: *solicitud de recursos* (desde Jefe de faena).

Salidas: *recursos disponibles* (hacia Jefe de faena).

Almacenes: D3 Trabajadores y D6 Programaciones (lectura).

Lógica:

1. **LEER** en D6 la fecha y la franja de la programación indicada.
2. **PARA CADA** cuadrilla de D3: **SI** todos sus trabajadores están habilitados y ninguno participa en otra programación de D6 que se cruce con la franja **ENTONCES** agregarla a las cuadrillas disponibles.
3. **PARA CADA** trabajador habilitado de D3 que pueda ser jefe tarjador: **SI** no dirige otra programación cruzada con la franja **ENTONCES** agregarlo a los jefes tarjadores disponibles.
4. **ENVIAR** *recursos disponibles*; **SI** ambas listas están vacías, indicar que la programación queda pendiente de asignación.

Proceso 3.2.2: Verificar disponibilidad

Entradas: *selección de recursos* (desde Jefe de faena).

Salidas: *asignación válida* (hacia 3.2.3); *rechazo de asignación* (hacia Jefe de faena).

Almacenes: D3 Trabajadores y D6 Programaciones (lectura).

Lógica:

1. **LEER** la franja de la programación en D6.
2. Verificar en D3 y D6 que la cuadrilla seleccionada siga disponible en la franja.
3. Verificar en D3 y D6 que el jefe tarjador esté habilitado y siga disponible.
4. **SI** alguno no está disponible **ENTONCES ENVIAR** *rechazo de asignación* indicando cuál y terminar.
5. **ENVIAR** *asignación válida*.

Proceso 3.2.3: Registrar asignación

Entradas: *asignación válida* (desde 3.2.2).

Salidas: *asignación registrada* (hacia Jefe de faena).

Almacenes: D6 Programaciones (lectura y escritura).

Lógica:

1. **ACTUALIZAR** en D6 la programación con id_cuadrilla e id_jefe_tarjador.
2. La faena queda disponible para el jefe tarjador, que la recibirá en el proceso 3.4.
3. **ENVIAR** *asignación registrada*.

Proceso 3.3: Modificar programación

Entradas: *cambio de programación* (desde Jefe de faena).

Salidas: *confirmación de programación*.

Almacenes: D6 Programaciones (lectura y escritura).

Lógica:

1. **LEER** la programación en D6; **SI** no existe **ENTONCES ENVIAR** el error y terminar.
2. **SI** acción = reprogramar **ENTONCES SI** estado $\neq$ PROGRAMADA o la nueva franja se cruza con otra programación **ENTONCES** rechazar **SINO ACTUALIZAR** fecha, hora_inicio y hora_termino.
3. **SI** acción = anular **ENTONCES SI** estado = PROGRAMADA **ENTONCES ACTUALIZAR** estado = ANULADA.
4. **SI** acción = eliminar **ENTONCES SI** estado = PROGRAMADA (aún sin operación) **ENTONCES** eliminarla de D6 **SINO** rechazar.
5. **SI** acción = modificar estado **ENTONCES ACTUALIZAR** estado al valor indicado.
6. **ENVIAR** *confirmación de programación* con el resultado.

Proceso 3.4: Entregar faenas asignadas

Entradas: *credenciales* (desde Jefe tarjador).

Salidas: *faenas asignadas* (hacia Jefe tarjador).

Almacenes: D1 Usuarios, D5 Manifiestos y D6 Programaciones (lectura).

Lógica:

1. **LEER** D1; **SI** el email no existe, la password no coincide o activo = falso **ENTONCES ENVIAR**

"credenciales inválidas" y terminar.
2. Obtener el id_trabajador ligado al usuario.
3. **LEER** D6: programaciones con id_jefe_tarjador = id_trabajador y estado PROGRAMADA o EN_EJECUCION, ordenadas por fecha y hora_inicio.
4. **PARA CADA** programación: **LEER** en D5 el contenedor y sus líneas declaradas.
5. **ENVIAR** *faenas asignadas*, aplicando el filtro de búsqueda si se indicó.

4.3.4. Proceso 4: Ejecutar operación en terreno

Todos los procesos que reciben una imagen aplican la misma **regla de evidencia**: no aceptan el registro sin una imagen válida y escriben en D8 una fotografía con uuid, etapa, fecha_hora_captura, latitud, longitud, estado = PENDIENTE e id_operacion.

Proceso 4.1: Iniciar operación

Entradas: *datos de inicio* (desde Jefe tarjador).

Salidas: *confirmación de registro* (hacia Jefe tarjador).

Almacenes: D5 Manifiestos (lectura); D6 Programaciones (lectura y escritura); D7 Operaciones (escritura).

Lógica:

1. **LEER** la programación en D6; **SI** no está asignada a este jefe tarjador o su estado no es PROGRAMADA **ENTONCES ENVIAR** el error y terminar.
2. **LEER** en D5 las líneas declaradas del contenedor.
3. **SI** existen líneas **ENTONCES** modalidad_ingreso = MANIFIESTO_XML **SINO** modalidad_ingreso = TARJA_CIEGA.
4. **ESCRIBIR** en D7 la operación con hora_inicio = ahora, estado = EN_EJECUCION.
5. **ACTUALIZAR** en D6 estado = EN_EJECUCION.
6. **ENVIAR** *confirmación de registro* con el contenedor, sus líneas y la modalidad.

Proceso 4.2: Verificar sello

Entradas: *datos de sello* (desde Jefe tarjador).

Salidas: *confirmación de registro*.

Almacenes: D5 Manifiestos (lectura); D7 Operaciones y D8 Fotografías (escritura).

Lógica:

1. Aplicar la regla de evidencia con etapa = SELLO.
2. **LEER** en D5 el nro_sello_declarado del contenedor.
3. **SI** el sello está roto **ENTONCES** estado = ROTO **SINO SI** numero_digitado $\neq$ nro_sello_declarado **ENTONCES** estado = NO_COINCIDE **SINO** estado = INTACTO.
4. **ESCRIBIR** en D7 el sello con numero_digitado, estado y fecha_verificacion.
5. **SI** estado $\neq$ INTACTO **ENTONCES ENVIAR** *confirmación de registro* exigiendo registrar una incidencia SELLO_ALTERADO antes de la apertura **SINO ENVIAR** *confirmación de registro* habilitando la fotografía de apertura (etapa = APERTURA).

Proceso 4.3.1: Identificar línea de carga

Entradas: *datos de lote* (desde Jefe tarjador).

Salidas: *lote identificado* (hacia 4.3.2).

Almacenes: D2 Clientes y D4 Tipos de bulto (lectura); D5 Manifiestos (lectura y escritura).

Lógica:

1. **SI** la operación es MANIFIESTO_XML **ENTONCES LEER** en D5 la línea indicada; **SI** no pertenece al contenedor **ENTONCES** rechazar el lote.
2. **SINO** (tarja ciega):
   a. **LEER** D2 y D4 y verificar que el cliente y el tipo de bulto existan y estén activos.
   b. **ESCRIBIR** en D5 una línea nueva con descripcion, id_cliente, id_tipo_bulto y cantidad_declarada = cantidad recibida.
3. **ENVIAR** *lote identificado* con id_linea, cantidad recibida, imagen e indicador de daño visible.

Proceso 4.3.2: Contrastar cantidad

Entradas: *lote identificado* (desde 4.3.1).

Salidas: *diferencia de cantidad* (hacia 4.3.3).

Almacenes: D5 Manifiestos (lectura y escritura); D8 Fotografías (escritura).

Lógica:

1. Aplicar la regla de evidencia con etapa = LOTE e id_linea.
2. **ACTUALIZAR** en D5 cantidad_recibida = cantidad_recibida + cantidad del lote.
3. diferencia = cantidad_recibida $-$ cantidad_declarada (0 en tarja ciega).
4. **ENVIAR** *diferencia de cantidad* con diferencia e indicador de daño visible.

Proceso 4.3.3: Evaluar diferencia

Entradas: *diferencia de cantidad* (desde 4.3.2).

Salidas: *confirmación de registro* (hacia Jefe tarjador).

Almacenes: Ninguno.

Lógica:

1. **SI** diferencia > 0 **ENTONCES** tipo sugerido = SOBRANTE.
2. **SI** se informó daño visible **ENTONCES** tipo sugerido = DANO.
3. **SI** existe tipo sugerido **ENTONCES ENVIAR** *confirmación de registro* exigiendo registrar la incidencia sugerida antes del siguiente lote **SINO ENVIAR** *confirmación de registro* con el lote confirmado.
4. Los faltantes se determinan al cerrar la faena (proceso 4.5), cuando ya no pueden llegar más lotes de la línea.

Proceso 4.4: Registrar incidencia

Entradas: *datos de incidencia* (desde Jefe tarjador).

Salidas: *confirmación de registro*.

Almacenes: D7 Operaciones y D8 Fotografías (escritura).

Lógica:

1. **SI** tipo $\neq$ SELLO_ALTERADO y no se indica id_linea **ENTONCES** rechazar.
2. **SI** no se adjunta al menos una imagen **ENTONCES** rechazar (evidencia obligatoria).
3. **ESCRIBIR** en D7 la incidencia con tipo, descripcion, cantidad_afectada, fecha_hora, id_operacion e id_linea.
4. **PARA CADA** imagen: aplicar la regla de evidencia con etapa = INCIDENCIA e id_incidencia.
5. **ENVIAR** *confirmación de registro* habilitando la continuación del flujo.

Proceso 4.5: Cerrar faena

Entradas: *datos de cierre* (desde Jefe tarjador).

Salidas: *confirmación de registro* (hacia Jefe tarjador); *operación finalizada* (hacia proceso 5).

Almacenes: D5 Manifiestos (lectura); D6 Programaciones, D7 Operaciones y D8 Fotografías (lectura y escritura).

Lógica:

1. Aplicar la regla de evidencia con etapa = CIERRE.
2. **LEER** D8; **SI** falta alguna de las etapas SELLO, APERTURA o CIERRE **ENTONCES ENVIAR** las etapas faltantes y terminar.
3. **PARA CADA** línea del contenedor en D5 con cantidad_recibida $<$ cantidad_declarada:
   a. Solicitar confirmación al Jefe tarjador; **SI** confirma **ENTONCES ESCRIBIR** en D7 una incidencia FALTANTE por la diferencia **SINO** terminar.
4. **ACTUALIZAR** en D7 la operación con hora_fin = ahora y estado = FINALIZADA, y en D6 la programación con estado = FINALIZADA.
5. **ENVIAR** *operación finalizada* con el id_operacion y **ENVIAR** *confirmación de registro* con el cierre de la faena.

4.3.5. Proceso 5: Validar reporte

Proceso 5.1: Generar borrador de reporte

Entradas: *operación finalizada* (desde proceso 4); *evidencia corregida* (desde 5.5).

Salidas: Ninguna hacia terminadores.

Almacenes: D5 Manifiestos, D7 Operaciones y D8 Fotografías (lectura); D9 Reportes (escritura).

Lógica:

1. **LEER** en D7 la operación, su sello y sus incidencias; en D5 el contenedor y sus líneas con su cliente;

y en D8 las fotografías no rechazadas ni reemplazadas.
2. Ordenar las fotografías por etapa y, dentro de la etapa LOTE, por cliente.
3. Componer el documento PDF con los datos, las incidencias y las fotografías.
4. **SI** la entrada es *operación finalizada* **ENTONCES ESCRIBIR** en D9 el reporte con folio nuevo, version = 1, estado = BORRADOR, fecha_generacion = ahora y estado_envio = PENDIENTE.
5. **SINO ACTUALIZAR** en D9 version = version + 1, estado = BORRADOR y el nuevo documento.

Proceso 5.2: Presentar reporte

Entradas: *solicitud de revisión* (desde Supervisor del reporte).

Salidas: *reporte para revisión* (hacia Supervisor del reporte).

Almacenes: D9 Reportes (lectura).

Lógica:

1. **SI** no se indica un reporte **ENTONCES LEER** D9 y **ENVIAR** la bandeja de reportes en estado BORRADOR con folio, operación y fecha; terminar.
2. **LEER** en D9 el reporte indicado; **SI** su estado no es BORRADOR **ENTONCES** informarlo y terminar.
3. **ENVIAR** *reporte para revisión* con los datos de la faena y las fotografías ordenadas por etapa y cliente.

Proceso 5.3: Aprobar reporte

Entradas: *decisión de validación* con aprobación (desde Supervisor).

Salidas: *reporte aprobado* (hacia proceso 6); *reporte para revisión* con aviso (hacia Supervisor).

Almacenes: D8 Fotografías (lectura); D9 Reportes (lectura y escritura).

Lógica:

1. **LEER** D8; **SI** falta evidencia obligatoria o hay fotografías RECHAZADAS sin reemplazo **ENTONCES ENVIAR** *reporte para revisión* con el aviso de la etapa que carece de fotografías y terminar.
2. **ACTUALIZAR** en D9 estado = APROBADO, fecha_aprobacion = ahora e id_aprobador = usuario supervisor.
3. **ENVIAR** *reporte aprobado* con el id_reporte.

Proceso 5.4: Rechazar evidencia

Entradas: *decisión de validación* con rechazo (desde Supervisor).

Salidas: *solicitud de recaptura* (hacia Jefe tarjador).

Almacenes: D8 Fotografías y D9 Reportes (lectura y escritura).

Lógica:

1. **SI** no se indica motivo_rechazo **ENTONCES** solicitarlo y terminar.

2. **PARA CADA** fotografía marcada: **ACTUALIZAR** en D8 estado = RECHAZADA y motivo_rechazo.
3. **ACTUALIZAR** en D9 estado = EN_CORRECCION.
4. **ENVIAR** *solicitud de recaptura* con la operación, la etapa, la fotografía y el motivo.

Proceso 5.5: Reemplazar evidencia

Entradas: *fotografía de reemplazo* (desde Jefe tarjador).

Salidas: *evidencia corregida* (hacia 5.1).

Almacenes: D8 Fotografías (lectura y escritura).

Lógica:

1. **LEER** en D8 la fotografía rechazada; **SI** su estado no es RECHAZADA **ENTONCES** descartar el reemplazo y terminar.
2. **ESCRIBIR** en D8 la nueva fotografía con la misma etapa, operación, línea e incidencia que la rechazada.
3. **ACTUALIZAR** la fotografía rechazada con estado = REEMPLAZADA.
4. **SI** no quedan fotografías RECHAZADAS en la operación **ENTONCES ENVIAR** *evidencia corregida*.

4.3.6. Proceso 6: Distribuir y consultar reportes

Proceso 6.1: Enviar reporte

Entradas: *reporte aprobado* (desde proceso 5); *solicitud de reenvío* (desde Jefe de faena).

Salidas: *reporte PDF* (hacia Cliente del depósito); *resultado de envío* (hacia Supervisor o Jefe de faena).

Almacenes: D2 Clientes y D5 Manifiestos (lectura); D9 Reportes (lectura y escritura).

Lógica:

1. **LEER** el reporte en D9; **SI** su estado no es APROBADO **ENTONCES ENVIAR** *resultado de envío* "reporte pendiente de validación" y terminar.
2. **LEER** en D5 los clientes que tienen líneas en el contenedor de la operación y, **PARA CADA** uno, **LEER** su email en D2.
3. **SI** ningún cliente tiene email **ENTONCES** resultado = FALLIDO con el motivo "completar ficha del cliente" **SINO ENVIAR** *reporte PDF* a los destinatarios y resultado = ENVIADO o FALLIDO según la respuesta del envío.
4. **ESCRIBIR** en D9 un envío con fecha_hora, destinatarios, origen (AUTOMATICO si viene de 5.3, REENVIO si lo pidió el Jefe de faena) y resultado; **ACTUALIZAR** estado_envio del reporte.
5. **ENVIAR** *resultado de envío* al Supervisor (origen automático) o al Jefe de faena (reenvío).

Proceso 6.2: Consultar operaciones

Entradas: *consulta de operaciones* (desde Jefe de faena).

Salidas: *listado de operaciones* (hacia Jefe de faena).

Almacenes: D5 Manifiestos, D6 Programaciones y D9 Reportes (lectura); D7 Operaciones (lectura y escritura).

Lógica:

1. **SI** la consulta incluye un cambio de estado **ENTONCES ACTUALIZAR** en D7 el estado de la operación indicada (no se permite volver a EN_EJECUCION una operación con reporte aprobado).
2. **LEER** D7, D6 y D5 y seleccionar las operaciones que cumplen los filtros (rango de fechas, cliente, número de contenedor, estado).
3. **PARA CADA** operación: **LEER** en D9 el estado del reporte y su estado_envio.
4. **ENVIAR** *listado de operaciones*; **SI** está vacío, indicar "sin resultados".

Proceso 6.3: Consultar estado del cliente

Entradas: *consulta de estado* (desde Cliente del depósito).

Salidas: *estado de operaciones* (hacia Cliente del depósito).

Almacenes: D1 Usuarios, D5 Manifiestos, D6 Programaciones, D7 Operaciones y D9 Reportes (lectura).

Lógica:

1. **LEER** D1; **SI** las credenciales son inválidas o la cuenta no está ligada a un cliente **ENTONCES** rechazar el acceso y terminar.
2. **LEER** en D5 los contenedores con al menos una línea cuyo id_cliente es el del usuario.
3. **PARA CADA** contenedor que cumple los filtros: **LEER** su programación en D6, su operación en D7 y el estado de su reporte en D9.
4. **ENVIAR** *estado de operaciones*, indicando cuáles tienen reporte APROBADO disponible para descarga.

Proceso 6.4: Entregar reporte PDF

Entradas: *solicitud de PDF* (desde Cliente del depósito).

Salidas: *reporte PDF* (hacia Cliente del depósito).

Almacenes: D9 Reportes (lectura).

Lógica:

1. **LEER** el reporte en D9; **SI** no existe o no está APROBADO **ENTONCES** rechazar y terminar. El cliente solo puede solicitar reportes que el proceso 6.3 le mostró como disponibles, es decir, de operaciones con carga consignada a su empresa.
2. **SI** modo = visualizar **ENTONCES ENVIAR** *reporte PDF* para mostrarlo en pantalla **SINO ENVIAR** *reporte PDF* como archivo descargable.

4.4. Diccionario de datos

El diccionario de datos define de forma rigurosa cada flujo, almacén y dato elemental que aparece en los DFD y en las mini especificaciones (Yourdon, 1989). Es la referencia que garantiza el balanceo entre niveles: un flujo compuesto del nivel superior se define aquí como la selección o concatenación de los flujos en que se descompone.

4.4.1. Notación

Se usa la notación de DeMarco (1979) resumida en la Tabla 2.

Tabla 2: Notación del diccionario de datos.

Símbolo	Significado	Ejemplo
`=`	está compuesto de	`credenciales = email + password`
`+`	y (concatenación)	`nave + viaje`
`[ a | b ]`	selección de una alternativa	`[aprobación | rechazo]`
`{ a }`	iteración (cero o más veces)	`{imagen}`
`n{ a }m`	iteración entre n y m veces	`1{imagen}` (al menos una)
`( a )`	dato opcional	`(id_linea)`
`@`	identificador (clave) de un almacén	`@id_usuario`
`* texto *`	comentario	`* solo en tarja ciega *`

4.4.2. Flujos de datos

La Tabla 3 define los flujos de todos los niveles en orden alfabético. Los flujos del diagrama de contexto aparecen como selecciones de los flujos de nivel inferior.

Tabla 3: Diccionario de datos - Flujos de datos.

Flujo	Composición	Origen → destino
asignación registrada	id_programacion + id_cuadrilla + id_jefe_tarjador	3.2.3 → Jefe de faena
asignación válida	id_programacion + id_cuadrilla + id_jefe_tarjador	3.2.2 → 3.2.3
aviso de importación duplicada	nave + viaje + fecha_recepcion	2.1 → Jefe de faena
cambio de programación	id_programacion + accion_programacion + (fecha + hora_inicio + hora_termino) + (estado_faena)	Jefe de faena → 3.3
confirmación de programación	[{contenedor pendiente} | id_programacion + estado_faena | recursos disponibles | rechazo de asignación | asignación registrada] + (mensaje) * contenedor pendiente = id_contenedor + numero + fecha_arribo + {descripcion + cantidad_declarada} *	3.1, 3.2, 3.3 → Jefe de faena
confirmación de registro	id_operacion + resultado + (mensaje) + (tipo_incidencia * incidencia exigida *) + ({linea de faena})	4.1 a 4.5, 4.3.3 → Jefe tarjador

Flujo	Composición	Origen → destino
consulta de estado	credenciales + (fecha_desde + fecha_hasta) + (numero)	Cliente del depósito → 6.3
consulta de operaciones	filtros + (id_operacion + estado_faena * cambio de estado *)	Jefe de faena → 6.2
contenedores declarados	id_manifiesto + 1{numero + tamano + nro_sello_declarado + fecha_arribo + {linea declarada}}	2.3.1 → 2.3.2
credenciales	email + password	Jefe tarjador → 3.4; parte de consulta de estado
datos de asignación	[solicitud de recursos | selección de recursos]	Jefe de faena → 3.2
datos de cierre	id_operacion + imagen + (confirma_faltantes)	Jefe tarjador → 4.5
datos de cliente	accion + (id_cliente) + rut + razon_social + email + (activo)	Administrador → 1.4
datos de incidencia	id_operacion + tipo_incidencia + (id_linea) + descripcion + cantidad_afectada + 1{imagen}	Jefe tarjador → 4.4
datos de inicio	id_programacion	Jefe tarjador → 4.1
datos de lote	id_operacion + [id_linea | descripcion + id_cliente + id_tipo_bulto * tarja ciega *] + cantidad_recibida + dano_visible + imagen	Jefe tarjador → 4.3 / 4.3.1
datos de nueva programación	[listar_pendientes | [id_contenedor | numero + tamano + nro_sello_declarado * tarja ciega *] + fecha + hora_inicio + hora_termino]	Jefe de faena → 3.1
datos de programación	[datos de nueva programación | datos de asignación | cambio de programación]	Jefe de faena → 3
datos de rol	accion + (id_rol) + nombre + {codigo} + (activo)	Administrador → 1.2
datos de sello	id_operacion + numero_digitado + sello_roto + imagen	Jefe tarjador → 4.2
datos de tipo de bulto	accion + (id_tipo_bulto) + codigo + nombre + (activo)	Administrador → 1.3
datos de trabajador	accion + [(id_trabajador) + rut + nombre + cargo + (id_cuadrilla) + (habilitado) | (id_cuadrilla) + nombre * cuadrilla *]	Administrador → 1.5
datos de usuario	accion + (id_usuario) + nombre + email + (password) + id_rol + ([id_trabajador | id_cliente]) + (activo)	Administrador → 1.1
decisión de validación	id_reporte + [id_aprobador * aprobación * | 1{id_foto} + motivo_rechazo * rechazo *]	Supervisor → 5.3 / 5.4
diferencia de cantidad	id_linea + diferencia + dano_visible	4.3.2 → 4.3.3
errores de validación	nave + viaje + 1{error_validacion}	2.2 → 2.4
estado de operaciones	{numero + fecha + estado_faena + estado_reporte + (id_reporte)}	6.3 → Cliente del depósito
evidencia corregida	id_operacion	5.5 → 5.1
faenas asignadas	{id_programacion + fecha + hora_inicio + hora_termino + numero + nro_sello_declarado + {linea declarada}}	3.4 → Jefe tarjador
fotografía de reemplazo	id_foto + imagen	Jefe tarjador → 5.5

Flujo	Composición	Origen → destino
importación rechazada	nave + viaje	2.4 → 2.5
linea declarada	descripcion + cantidad_declarada + codigo + rut + razon_social	(dato compuesto de manifiesto XML)
líneas con consignatario	{linea declarada + id_contenedor + id_cliente} + cantidad_contenedores + {advertencia}	2.3.3 → 2.3.4
líneas declaradas	{linea declarada + id_contenedor} + cantidad_contenedores	2.3.2 → 2.3.3
listado de operaciones	{id_operacion + numero + fecha + estado_faena + (estado_reporte) + (estado_envio)}	6.2 → Jefe de faena
lote identificado	id_operacion + id_linea + cantidad_recibida + dano_visible + imagen	4.3.1 → 4.3.2
manifiesto válido	manifiesto XML + (reemplazar)	2.2 → 2.3 / 2.3.1
manifiesto XML	numero + nave + viaje + 1{numero + tamano + nro_sello_declarado + fecha_arribo + 1{linea declarada}}	Aduanas → 2.2
notificación de importación	nave + viaje + resultado	2.5 → Aduanas
operación finalizada	id_operacion	4.5 → 5.1
rechazo de asignación	id_programacion + mensaje	3.2.2 → Jefe de faena
recursos disponibles	id_programacion + {id_cuadrilla + nombre} + {id_trabajador + nombre}	3.2.1 → Jefe de faena
registro de terreno	[datos de inicio | datos de sello | datos de lote | datos de incidencia | datos de cierre]	Jefe tarjador → 4
reporte aprobado	id_reporte	5.3 → 6.1
reporte de errores	nave + viaje + fecha + 1{error_validacion} + (mensaje * sugerencia *)	2.4 → Jefe de faena
reporte para revisión	[{id_reporte + folio + id_operacion + fecha_generacion} * bandeja * | id_reporte + folio + version + documento_pdf] + (mensaje * evidencia faltante *)	5.2, 5.3 → Supervisor
reporte PDF	folio + documento_pdf + (modo)	6.1, 6.4 → Cliente del depósito
resultado de envío	id_reporte + resultado_envio + destinatarios + (mensaje)	6.1 → Supervisor / Jefe de faena
resultado de importación	[resumen de importación | reporte de errores | aviso de importación duplicada]	2 → Jefe de faena
resultado de mantención	resultado + (id_registro) + (mensaje)	1.1 a 1.5 → Administrador
resumen de importación	nave + viaje + cantidad_contenedores + cantidad_lineas + {advertencia}	2.5 → Jefe de faena
resumen de registro	cantidad_contenedores + cantidad_lineas + {advertencia}	2.3.4 → 2.5
selección de recursos	id_programacion + id_cuadrilla + id_jefe_tarjador	Jefe de faena → 3.2.2
solicitud de importación	nave + viaje + fecha + (reemplazar)	Jefe de faena → 2.1

Flujo	Composición	Origen → destino
solicitud de manifiesto	nave + viaje + fecha	2.1 → Aduanas
solicitud de mantención	[datos de usuario | datos de rol | datos de tipo de bulto | datos de cliente | datos de trabajador]	Administrador → 1
solicitud de PDF	id_reporte + modo	Cliente del depósito → 6.4
solicitud de recaptura	id_operacion + id_foto + etapa + motivo_rechazo	5.4 → Jefe tarjador
solicitud de recursos	id_programacion	Jefe de faena → 3.2.1
solicitud de reenvío	id_reporte	Jefe de faena → 6.1
solicitud de revisión	(id_reporte)	Supervisor → 5.2
viaje solicitado	nave + viaje + fecha + (reemplazar)	2.1 → 2.2

4.4.3. Almacenes de datos

Cada almacén se define como un conjunto de registros. Los registros coinciden con las entidades del DER y con las tablas del modelo relacional (Tabla 4).

Tabla 4: Diccionario de datos - Almacenes.

Id	Almacén	Composición
D1	Usuarios	{usuario} + {rol} + {permiso} + {rol_permiso} usuario = @id_usuario + nombre + email + password_hash + activo + id_rol + (id_trabajador) + (id_cliente) rol = @id_rol + nombre + activo permiso = @id_permiso + codigo + descripcion rol_permiso = @id_rol + @id_permiso
D2	Clientes	{cliente} cliente = @id_cliente + rut + razon_social + email + activo
D3	Trabajadores	{trabajador} + {cuadrilla} trabajador = @id_trabajador + rut + nombre + cargo + habilitado + (id_cuadrilla) cuadrilla = @id_cuadrilla + nombre
D4	Tipos de bulto	{tipo_bulto} tipo_bulto = @id_tipo_bulto + codigo + nombre + activo
D5	Manifiestos	{manifiesto} + {contenedor} + {linea_carga} manifiesto = @id_manifiesto + numero + nave + viaje + fecha_recepcion contenedor = @id_contenedor + numero + tamano + nro_sello_declarado + fecha_arribo + (id_manifiesto) linea_carga = @id_linea + descripcion + cantidad_declarada + cantidad_recibida + id_contenedor + id_cliente + id_tipo_bulto
D6	Programaciones	{programacion} programacion = @id_programacion + fecha + hora_inicio + hora_termino + estado_faena + id_contenedor + (id_cuadrilla) + (id_jefe_tarjador)

Id	Almacén	Composición
D7	Operaciones	{operacion} + {sello_seguridad} + {incidencia} operacion = @id_operacion + hora_inicio + (hora_fin) + estado_faena + modalidad_ingreso + id_programacion sello_seguridad = @id_sello + numero_digitado + estado_sello + fecha_verificacion + id_operacion incidencia = @id_incidencia + tipo_incidencia + descripcion + cantidad_afectada + fecha_hora + id_operacion + (id_linea)
D8	Fotografías	{fotografia} fotografia = @id_foto + uuid + etapa + fecha_hora_captura + latitud + longitud + (url) + estado_foto + (motivo_rechazo) + id_operacion + (id_linea) + (id_incidencia)
D9	Reportes	{reporte} + {envio_reporte} reporte = @id_reporte + folio + version + estado_reporte + fecha_generacion + (fecha_aprobacion) + estado_envio + documento_pdf + id_operacion + (id_aprobador) envio_reporte = @id_envio + fecha_hora + destinatarios + origen_envio + resultado_envio + id_reporte

4.4.4. Datos elementales

La Tabla 5 define los datos que no se descomponen. Los identificadores de todas las entidades (id_usuario, id_rol, id_permiso, id_cliente, id_trabajador, id_cuadrilla, id_tipo_bulto, id_manifiesto, id_contenedor, id_linea, id_programacion, id_operacion, id_sello, id_incidencia, id_foto, id_reporte, id_envio) son enteros positivos asignados por el sistema; id_jefe_tarjador e id_aprobador son alias de id_trabajador e id_usuario, respectivamente.

Tabla 5: Diccionario de datos - Datos elementales.

Dato	Tipo / dominio	Descripción
accion	[crear | editar | eliminar | activar | desactivar]	Operación de mantención solicitada sobre un dato maestro.
accion_programacion	[reprogramar | anular | eliminar | modificar_estado]	Cambio solicitado sobre una programación.
activo, habilitado	Lógico	Indica si el registro puede usarse en nuevas operaciones.
advertencia	Texto, hasta 200 caracteres	Aviso no bloqueante de la importación (por ejemplo, ficha de cliente incompleta).
cantidad_afectada	Entero ≥ 1	Bultos afectados por una incidencia.
cantidad_contenedores, cantidad_lineas	Entero ≥ 0	Totales registrados en una importación.
cantidad_declarada	Entero ≥ 1	Bultos declarados en el manifiesto para una línea.
cantidad_recibida	Entero ≥ 0	Bultos contados en terreno para una línea.
cargo	Texto, hasta 60 caracteres	Cargo del trabajador dentro de la cuadrilla.
codigo	Texto, hasta 20 caracteres	Código corto de un permiso o de un tipo de bulto.
confirma_faltantes	Lógico	Confirmación del cierre con líneas sin lote registrado.

Dato	Tipo / dominio	Descripción
dano_visible	Lógico	Indica que el lote presenta daño visible.
descripcion	Texto, hasta 250 caracteres	Descripción de la mercancía, del permiso o del hallazgo.
destinatarios	{email}	Direcciones a las que se envió el reporte.
diferencia	Entero	cantidad_recibida − cantidad_declarada.
documento_pdf	Archivo PDF	Reporte compilado con datos y fotografías.
email	Ver dominios	Correo de un usuario o de un cliente.
error_validacion	Texto + ubicación en el XML	Incumplimiento detectado al validar el manifiesto.
estado_envio	Ver dominios	Resultado del último envío del reporte.
estado_faena	Ver dominios	Estado compartido por programación y operación.
estado_foto	Ver dominios	Estado de una fotografía.
estado_reporte	Ver dominios	Estado de un reporte.
estado_sello	Ver dominios	Resultado de la verificación del sello.
etapa	Ver dominios	Etapa del desconsolidado a la que pertenece una fotografía.
fecha, fecha_arribo	Fecha (dd-mm-aaaa)	Fecha planificada o de arribo al terminal.
fecha_desde, fecha_hasta	Fecha	Rango de búsqueda de operaciones.
fecha_generacion, fecha_aprobacion, fecha_recepcion, fecha_verificacion, fecha_hora, fecha_hora_captura	Fecha y hora (dd-mm-aaaa hh:mm:ss)	Momento en que ocurrió el hecho registrado.
filtros	(fecha_desde + fecha_hasta) + (id_cliente) + (numero) + (estado_faena)	Criterios de búsqueda de operaciones.
folio	Texto, formato BT-aaaa-nnnnnn	Número correlativo del reporte.
hora_inicio, hora_termino, hora_fin	Hora (hh:mm) o fecha y hora	Límites de una franja o de una operación.
imagen	Archivo JPEG	Imagen capturada por la cámara del dispositivo.
latitud, longitud	Real, grados decimales	Geoposición de la captura de una fotografía.
listar_pendientes	Lógico	Indica que se solicita la lista de contenedores sin programación.
mensaje	Texto, hasta 250 caracteres	Explicación de un resultado o de un error.
modalidad_ingreso	Ver dominios	Origen de la carga de la operación.
modo	[visualizar | descargar]	Forma de entrega del reporte al cliente.
motivo_rechazo	Texto, hasta 250 caracteres	Observación del Supervisor al rechazar una fotografía.
nave, viaje	Texto, hasta 60 caracteres	Nombre de la nave e identificador de su viaje.

Dato	Tipo / dominio	Descripción
nombre	Texto, hasta 100 caracteres	Nombre de usuario, rol, trabajador, cuadrilla o tipo de bulto.
nro_sello_declarado, numero_digitado	Texto alfanumérico, hasta 20 caracteres	Número del sello declarado por el naviero y digitado en terreno.
numero	Texto	Número del manifiesto o del contenedor (ver dominios).
origen_envio	Ver dominios	Indica si el envío fue automático o un reenvío.
password	Texto, mínimo 8 caracteres con letras y números	Contraseña ingresada; nunca se almacena.
password_hash	Texto	Resumen criptográfico de la contraseña.
razon_social	Texto, hasta 150 caracteres	Razón social del cliente.
reemplazar	Lógico	Confirma el reemplazo de un manifiesto ya importado.
resultado	[exitoso | rechazado]	Resultado de una mantención, registro o importación.
resultado_envio	[ENVIADO | FALLIDO]	Resultado de un envío del reporte.
rut	Ver dominios	Rol único tributario de un cliente o trabajador.
sello_roto	Lógico	Indica que el sello se encontró físicamente roto.
tamano	[20 | 40]	Tamaño del contenedor en pies.
tipo_incidencia	Ver dominios	Clasificación de una incidencia.
url	Texto	Ubicación de la fotografía en el almacenamiento, una vez subida.
uuid	Texto de 36 caracteres	Identificador universal generado en el dispositivo.
version	Entero ≥ 1	Versión del reporte; aumenta con cada corrección.

4.4.5. Dominios y valores posibles

Formatos.

- `rut = 7{dígito}8 + "-" + [dígito | "K"]` (sin puntos; por ejemplo, `76543210-K`).
- `email = 1{carácter_permitido} + "@" + 1{carácter_permitido} + "." + 2{letra}`.
- `numero (contenedor) = 4{letra mayúscula}4 + 7{dígito}7` (norma ISO 6346; por ejemplo, `MSCU1234567`).
- `fecha = dd-mm-aaaa`; `hora = hh:mm`; `fecha_hora = fecha + hh:mm:ss`.

Valores de las enumeraciones (coinciden con los tipos enumerados del Informe 2).

- `estado_faena = [PROGRAMADA | EN_EJECUCION | FINALIZADA | ANULADA]`
- `modalidad_ingreso = [MANIFIESTO_XML | TARJA_CIEGA]`

- estado_sello = [INTACTO | ROTO | NO_COINCIDE]
- tipo_incidencia = [DANO | FALTANTE | SOBRANTE | MAL_EMBALAJE | SELLO_ALTERADO]
- etapa = [SELLO | APERTURA | LOTE | INCIDENCIA | CIERRE]
- estado_foto = [PENDIENTE | SUBIDA | RECHAZADA | REEMPLAZADA]
- estado_reporte = [BORRADOR | EN_CORRECCION | APROBADO]
- estado_envio = [PENDIENTE | ENVIADO | FALLIDO]
- origen_envio = [AUTOMATICO | REENVIO]

PARTE 2

DISEÑO ESTRUCTURADO

5. DIAGRAMA DE ESTRUCTURA

El diagrama de estructura traduce el modelo esencial del análisis en una arquitectura modular implementable: muestra los módulos del programa, la jerarquía de llamadas entre ellos y los datos que se intercambian en cada llamada (Page-Jones, 1988; Yourdon & Constantine, 1979). Se dibuja con la notación de Yourdon y Constantine (Figura 13):

- Cada **módulo** es un rectángulo; los **módulos predefinidos** (de biblioteca o de servicios externos ya existentes) llevan una doble barra vertical a cada lado.
- Una flecha de un módulo a otro es una **llamada**: el módulo superior invoca al subordinado y recupera el control cuando este termina.
- Las flechas cortas con un círculo vacío son **acoplamientos de datos** y las de círculo relleno son **acoplamientos de control** (banderas); su sentido indica si el dato baja hacia el subordinado o sube hacia el superior.
- El **rombo** en la base de un módulo indica una **selección**: según el tipo de transacción, se invoca solo uno de los módulos que parten de él.
- Un módulo con varias flechas de entrada tiene **fan-in**: se reutiliza desde varios módulos superiores y se dibuja una sola vez.

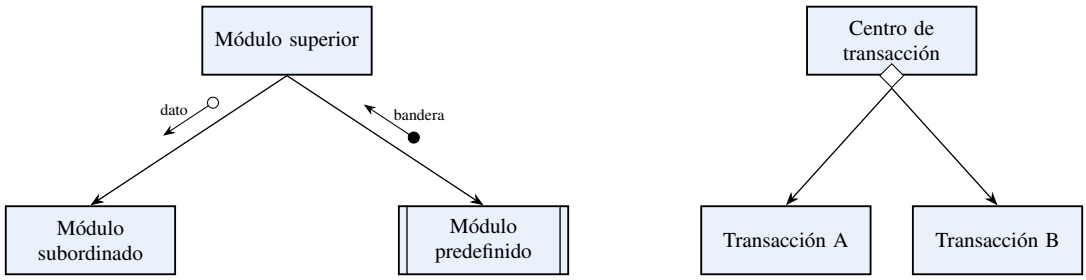


Figura 13: Notación del diagrama de estructura utilizada: llamada, acoplamiento de datos, bandera de control, módulo predefinido y selección.

5.1. Derivación desde los diagramas de flujo de datos

La arquitectura se obtuvo aplicando las dos estrategias clásicas del diseño estructurado sobre los DFD de la Parte 1:

- **Análisis de transacción.** El DFD de nivel 0 muestra que BeeTracer no procesa un único flujo, sino seis tipos de solicitud independientes, una por cada proceso principal. Por ello el módulo principal se diseñó como un *centro de transacción* (Figura 14): obtiene y autentica la solicitud, la despacha al subsistema que corresponde y muestra el resultado. Cada

subsistema equivale a un proceso del nivel 0 y conserva su nombre, lo que mantiene la trazabilidad entre análisis y diseño. Los subsistemas 1, 3, 4, 5 y 6 repiten el mismo patrón internamente, porque sus DFD de nivel 1 también reciben acontecimientos distintos que activan subprocesos distintos.

- **Análisis de transformación.** El subsistema Importar manifiesto (Figura 16) sí tiene un flujo principal único que se transforma por etapas, por lo que se dividió en una rama *aferente* (obtener el manifiesto válido: leer la solicitud, verificar duplicados, descargar el XML y validarlo), un *centro de transformación* (registrar el manifiesto, que corresponde al DFD de nivel 2 del proceso 2.3) y una rama *eferente* (emitir el resultado: reporte de errores, notificación a Aduanas y resumen).

Los procesos elementales de los DFD se convirtieron, en general, en módulos subordinados con el mismo nombre (por ejemplo, *Contrastar cantidad* y *Evaluar diferencia* provienen de los procesos 4.3.2 y 4.3.3). Además, aparecen módulos que no existían en el análisis porque corresponden a la **tecnología de implementación** que el modelo esencial excluyó: *Comprimir fotografía*, *Sellar fecha, hora y geoposición*, *Encolar fotografía* y *Sincronizar pendientes* resuelven el trabajo sin conexión en el patio; *Descargar XML de Aduanas* y *Notificar a Aduanas* encapsulan el web service del Servicio Nacional de Aduanas; *Enviar correo* encapsula el servidor de correo; y *Generar PDF*, *Calcular hash de contraseña*, *Autenticar usuario* y *Acceder a la base de datos* son servicios de biblioteca. Estos módulos corresponden a las clases de servicio del Informe 2 (`ServicioAduana`, `ServicioCorreo` y `SincronizadorOffline`).

5.2. Criterios de calidad del diseño

El diseño se evaluó con los criterios de acoplamiento y cohesión de Yourdon y Constantine (1979):

- **Acoplamiento bajo.** Los módulos se comunican casi exclusivamente mediante acoplamientos de datos (identificadores, imágenes, listas) y no comparten datos globales. Las banderas de control se limitan a informar resultados hacia arriba (*válido*, *disponible*, *completa*, *autenticado*), nunca a indicar al subordinado qué hacer, lo que evita el acoplamiento de control descendente.
- **Cohesión funcional.** Cada módulo realiza una sola función que puede describirse con un verbo y un objeto. Por ejemplo, *Verificar disponibilidad* solo verifica y no registra; el registro lo hace *Registrar asignación*.
- **Fan-in en lugar de duplicación.** La lógica que se repite en varios casos se concentra en un solo módulo reutilizado: *Registrar evidencia fotográfica* es invocado por los cuatro módulos que capturan fotografías, lo que garantiza que la regla de evidencia obligatoria se aplique de la misma forma en todas las etapas; *Guardar registro maestro*, *Verificar conflicto de agenda* y *Filtrar operaciones* cumplen el mismo papel en sus subsistemas.
- **Fan-out acotado.** Los módulos invocan a siete subordinados o menos, con una excepción justificada: *Administrar datos maestros* tiene ocho, pero cinco de ellos son alternativas excluyentes de una misma selección y en cada transacción solo se ejecuta una. Cuando un

subsistema crecía, se agregó un nivel intermedio (por ejemplo, *Obtener manifiesto válido* o *Registrar manifiesto*).

5.3. Diagramas

A continuación se presentan el panorama general y el diagrama de cada uno de los seis subsistemas. En las llamadas que se repiten con los mismos acoplamientos (por ejemplo, las cinco llamadas a *Guardar registro maestro*), estos se dibujan una sola vez.

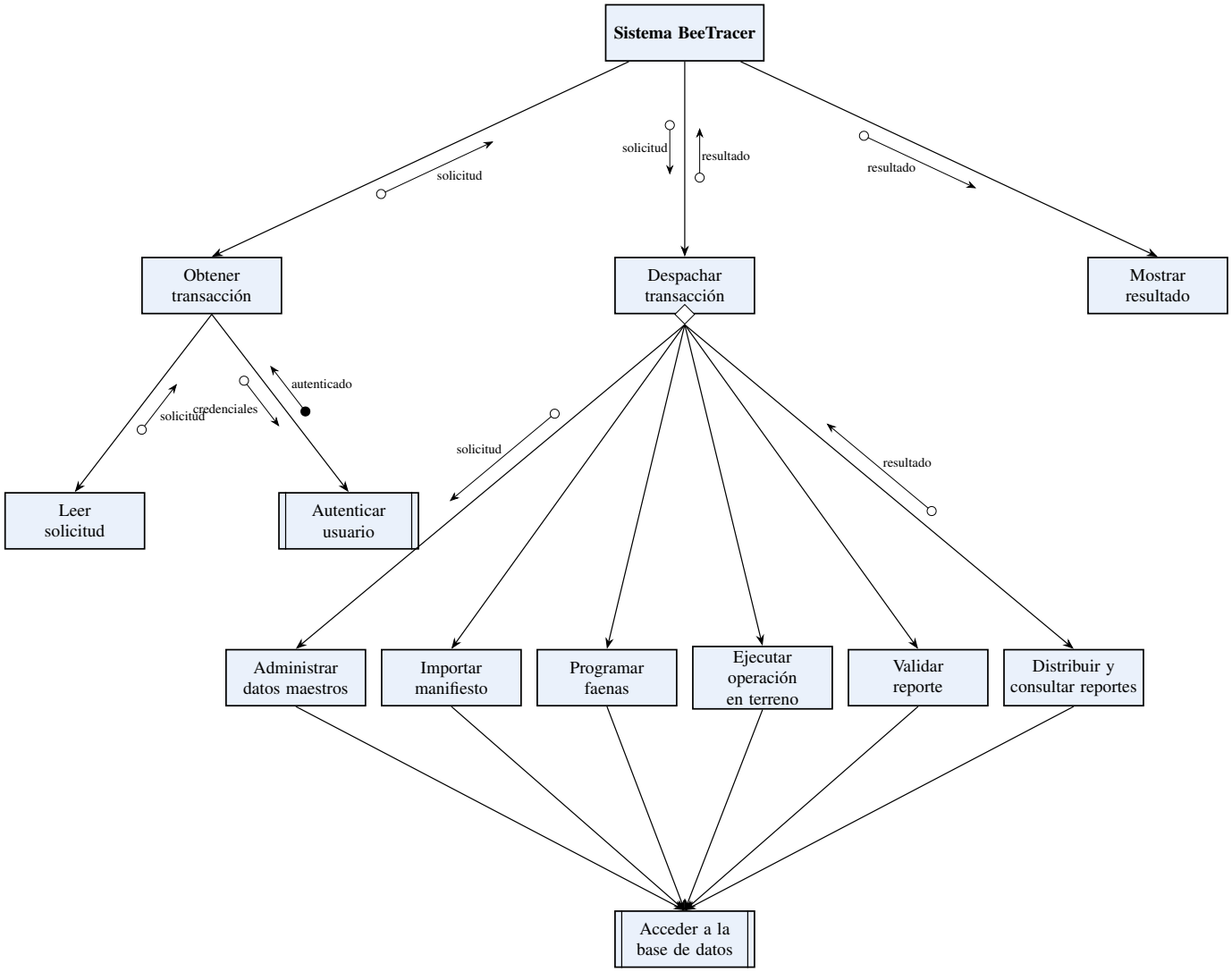


Figura 14: Diagrama de estructura - Panorama general (centro de transacción).

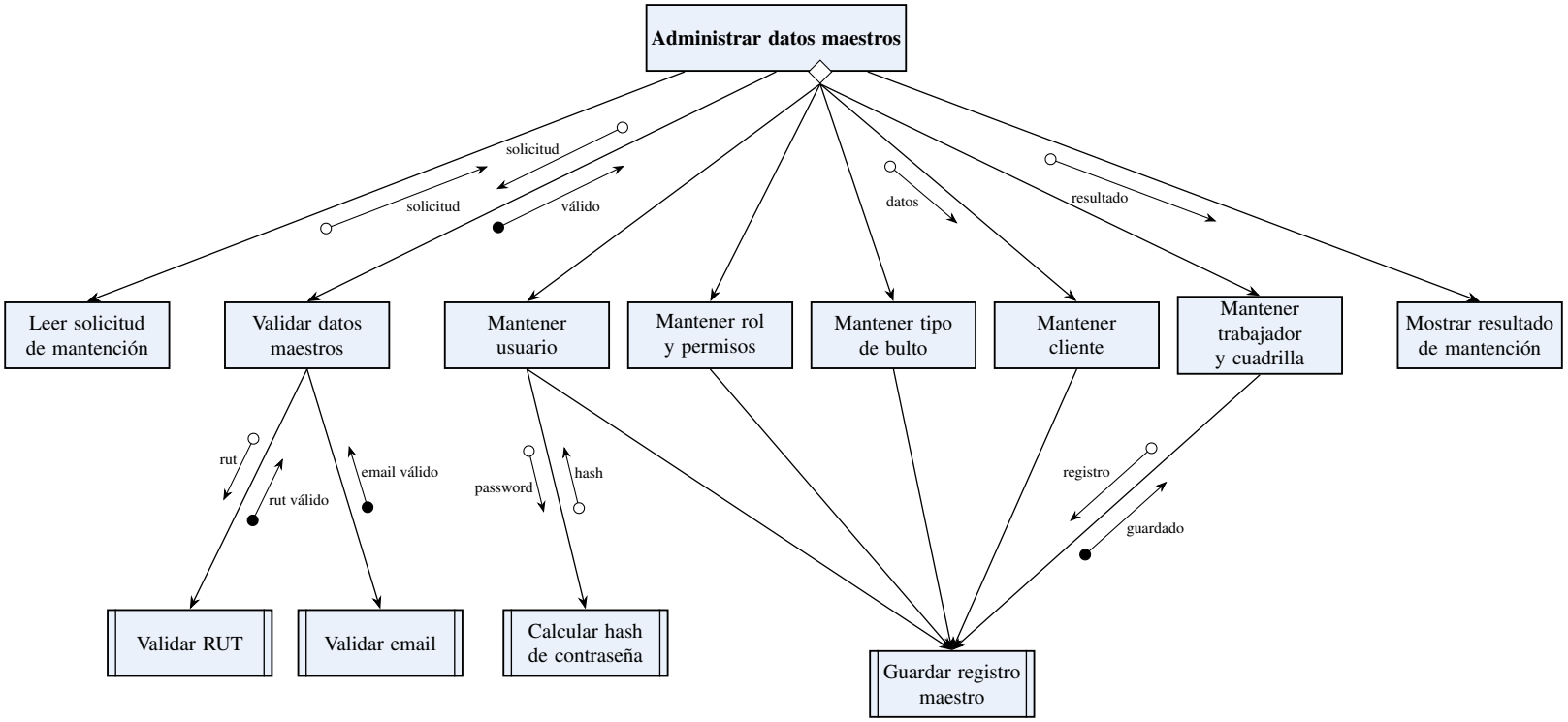


Figura 15: Diagrama de estructura - Subsistema 1: Administrar datos maestros.

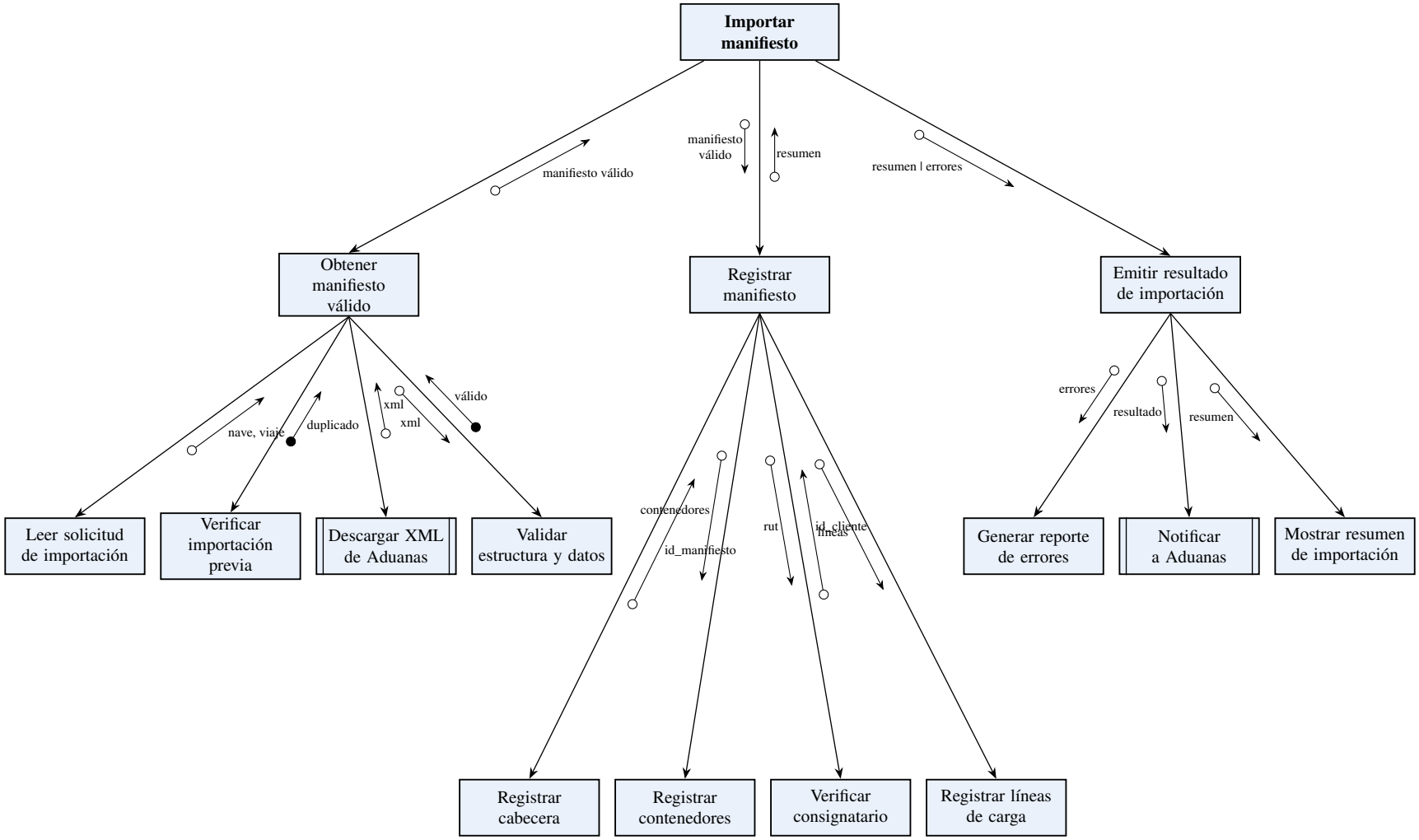


Figura 16: Diagrama de estructura - Subsistema 2: Importar manifiesto (análisis de transformación).

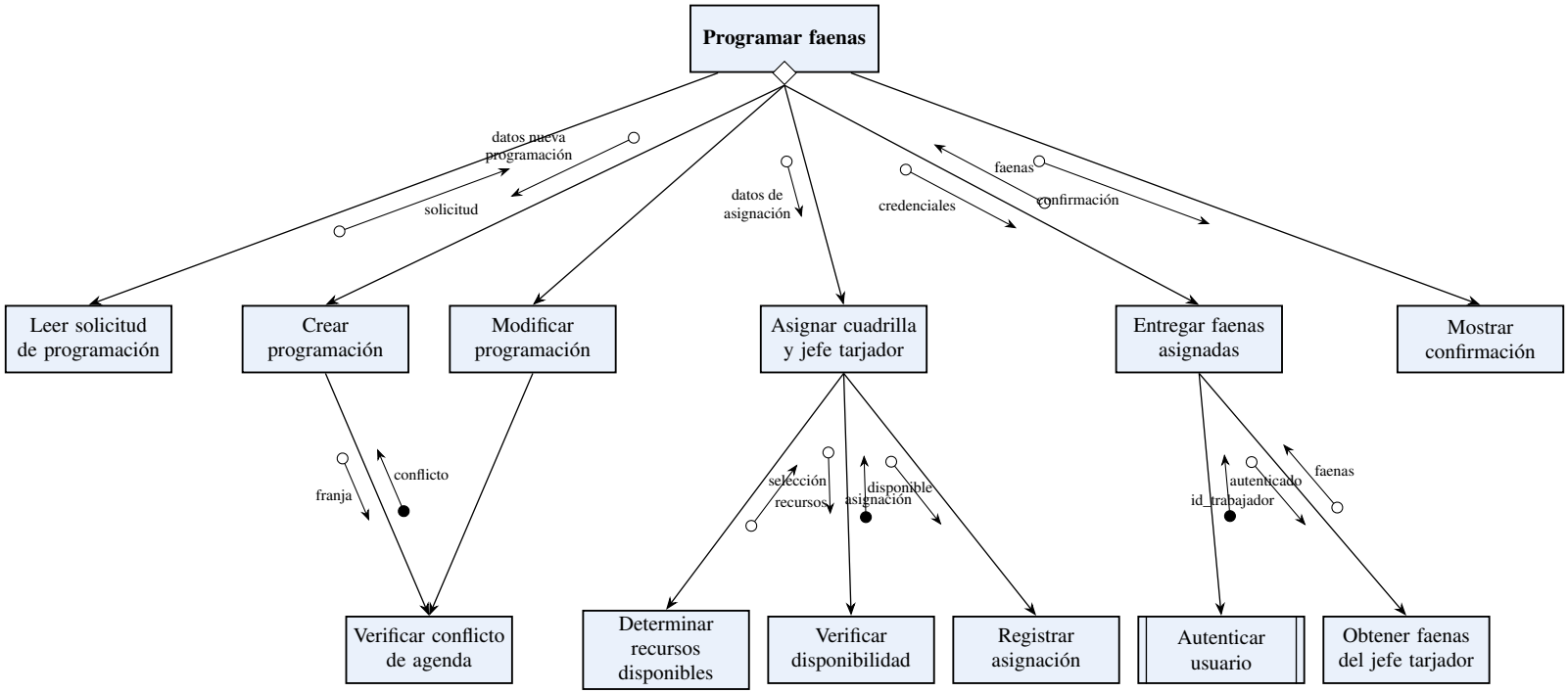


Figura 17: Diagrama de estructura - Subsistema 3: Programar faenas.

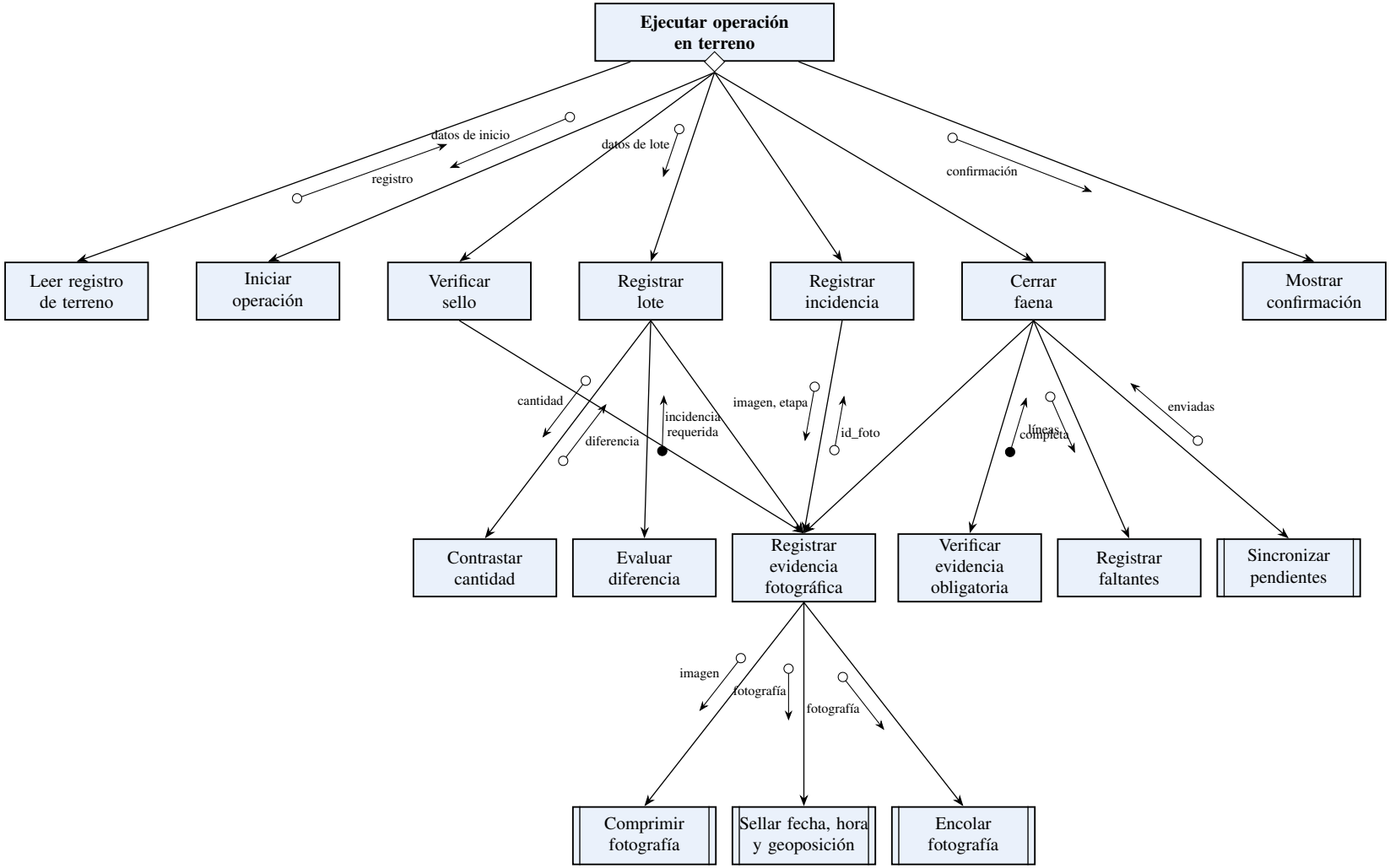


Figura 18: Diagrama de estructura - Subsistema 4: Ejecutar operación en terreno (aplicación móvil).

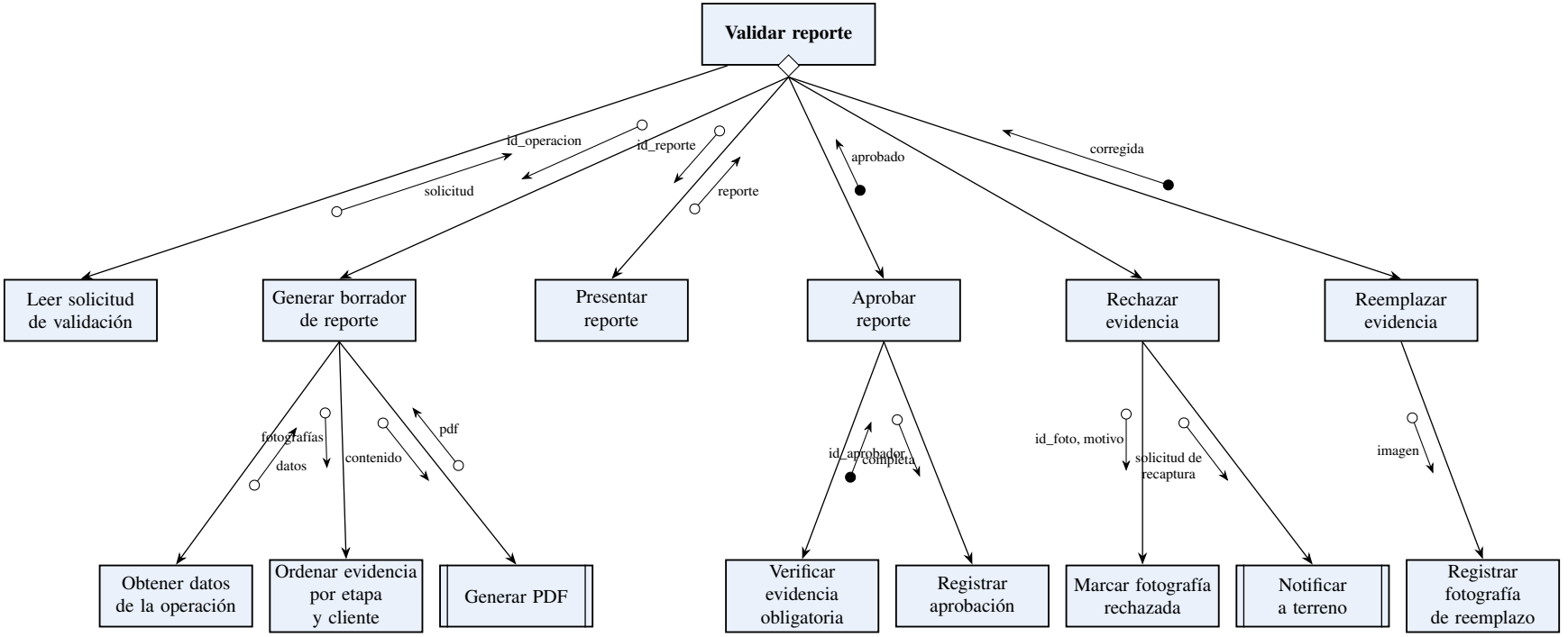


Figura 19: Diagrama de estructura - Subsistema 5: Validar reporte.

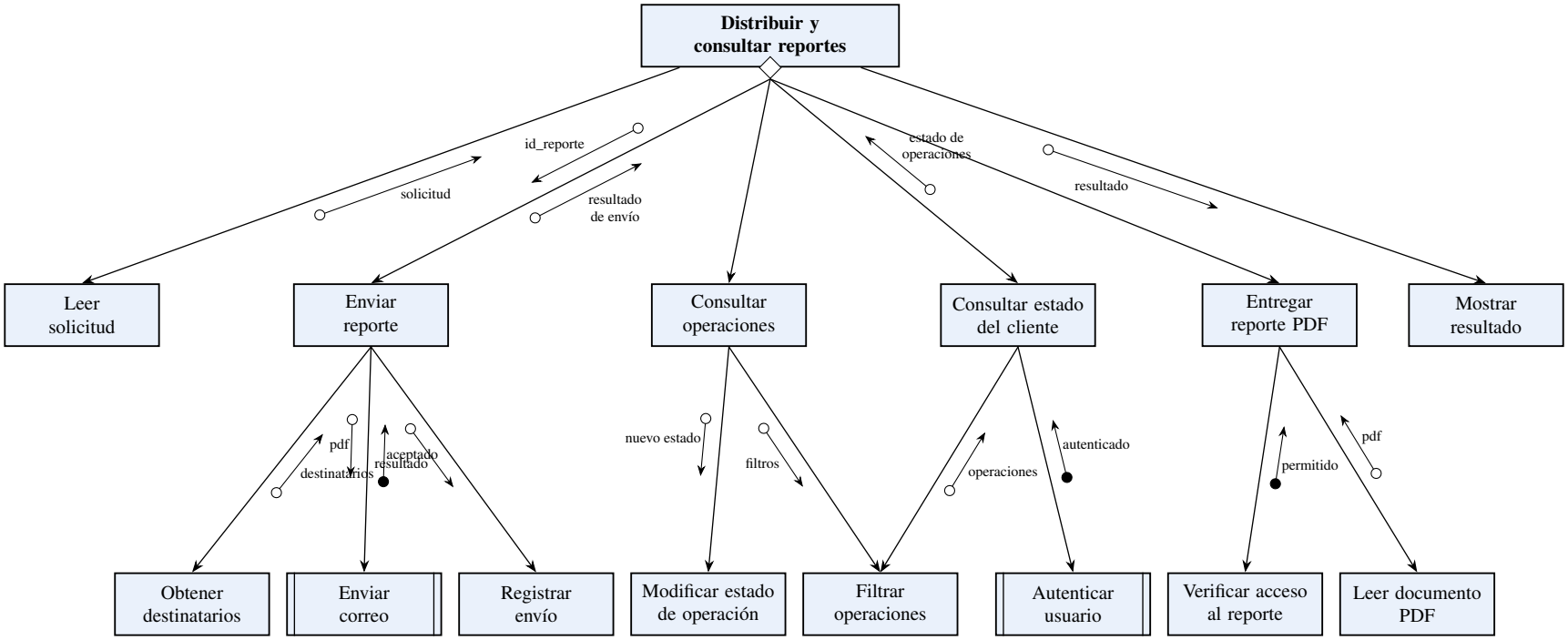


Figura 20: Diagrama de estructura - Subsistema 6: Distribuir y consultar reportes.

6. MODELO RELACIONAL

El modelo relacional traduce el diagrama entidad-relación (Sección 4.2) a un conjunto de tablas listas para implementarse en un motor de base de datos relacional (Codd, 1970; Elmasri & Navathe, 2016). Se aplicaron las reglas habituales de transformación:

1. Cada **entidad** se convierte en una tabla, con sus atributos como columnas y su identificador como clave primaria (PK).
2. Cada relación **1:N** se implementa agregando, en la tabla del lado N, una clave foránea (FK) que referencia a la clave primaria del lado 1 (por ejemplo, `LINEA_CARGA.id_contenedor`).
3. Cada relación **1:1** se implementa con una clave foránea con restricción de unicidad en la tabla dependiente (por ejemplo, `OPERACION.id_programacion` es FK y única, porque una programación origina una sola operación).
4. La relación **N:M** entre ROL y PERMISO se implementa con la tabla asociativa `ROL_PERMISO`, cuya clave primaria compuesta está formada por las dos claves foráneas.
5. Las **participaciones opcionales** del DER se implementan con claves foráneas que admiten nulo (por ejemplo, `PROGRAMACION.id_cuadrilla` mientras la faena no está asignada, o `CONTENEDOR.id_manifiesto` en tarja ciega).

6.1. Esquema relacional

A continuación se presenta el esquema en notación textual: la clave primaria va subrayada, las claves foráneas en *cursiva* con la tabla que referencian, y `?` indica que la columna admite nulo. La columna `estado` de cada tabla toma su dominio del diccionario de datos (`estado_faena`, `estado_sello`, `estado_foto` o `estado_reporte`, según corresponda).

- **ROL**(id_rol, nombre, activo)
- **PERMISO**(id_permiso, codigo, descripcion)
- **ROL_PERMISO**(*id_rol*→ROL, *id_permiso*→PERMISO)
- **CLIENTE**(id_cliente, rut, razon_social, email, activo)
- **CUADRILLA**(id_cuadrilla, nombre)
- **TRABAJADOR**(id_trabajador, rut, nombre, cargo, habilitado, *id_cuadrilla*?→CUADRILLA)
- **USUARIO**(id_usuario, nombre, email, password_hash, activo, *id_rol*→ROL, *id_trabajador*?→TRABAJADOR, *id_cliente*?→CLIENTE)
- **TIPO_BULTO**(id_tipo_bulto, codigo, nombre, activo)
- **MANIFIESTO**(id_manifiesto, numero, nave, viaje, fecha_recepcion)
- **CONTENEDOR**(id_contenedor, numero, tamano, nro_sello_declarado, fecha_arribo, *id_manifiesto*?→MANIFIEST

- **LINEA_CARGA**(id_linea, descripcion, cantidad_declarada, cantidad_recibida, *id_contenedor*→CONTENEDOR, *id_cliente*→CLIENTE, *id_tipo_bulto*→TIPO_BULTO)
- **PROGRAMACION**(id_programacion, fecha, hora_inicio, hora_termino, estado, *id_contenedor*→CONTENEDOR, *id_cuadrilla*?→CUADRILLA, *id_jefe_tarjador*?→TRABAJADOR)
- **OPERACION**(id_operacion, hora_inicio, hora_fin?, estado, modalidad_ingreso, *id_programacion*→PROGRAMAC
- **SELLO_SEGURIDAD**(id_sello, numero_digitado, estado, fecha_verificacion, *id_operacion*→OPERACION)
- **INCIDENCIA**(id_incidencia, tipo, descripcion, cantidad_afectada, fecha_hora, *id_operacion*→OPERACION, *id_linea*?→LINEA_CARGA)
- **FOTOGRAFIA**(id_foto, uuid, etapa, fecha_hora_captura, latitud, longitud, url?, estado, motivo_rechazo?, *id_operacion*→OPERACION, *id_linea*?→LINEA_CARGA, *id_incidencia*?→INCIDENCIA)
- **REPORTE**(id_reporte, folio, version, estado, fecha_generacion, fecha_aprobacion?, estado_envio, documento_pdf, *id_operacion*→OPERACION, *id_aprobador*?→USUARIO)
- **ENVIO_REPORTE**(id_envio, fecha_hora, destinatarios, origen, resultado, *id_reporte*→REPORTE)

La Figura 21 muestra las mismas tablas y sus claves foráneas; cada flecha va desde la tabla que contiene la clave foránea hacia la tabla referenciada.

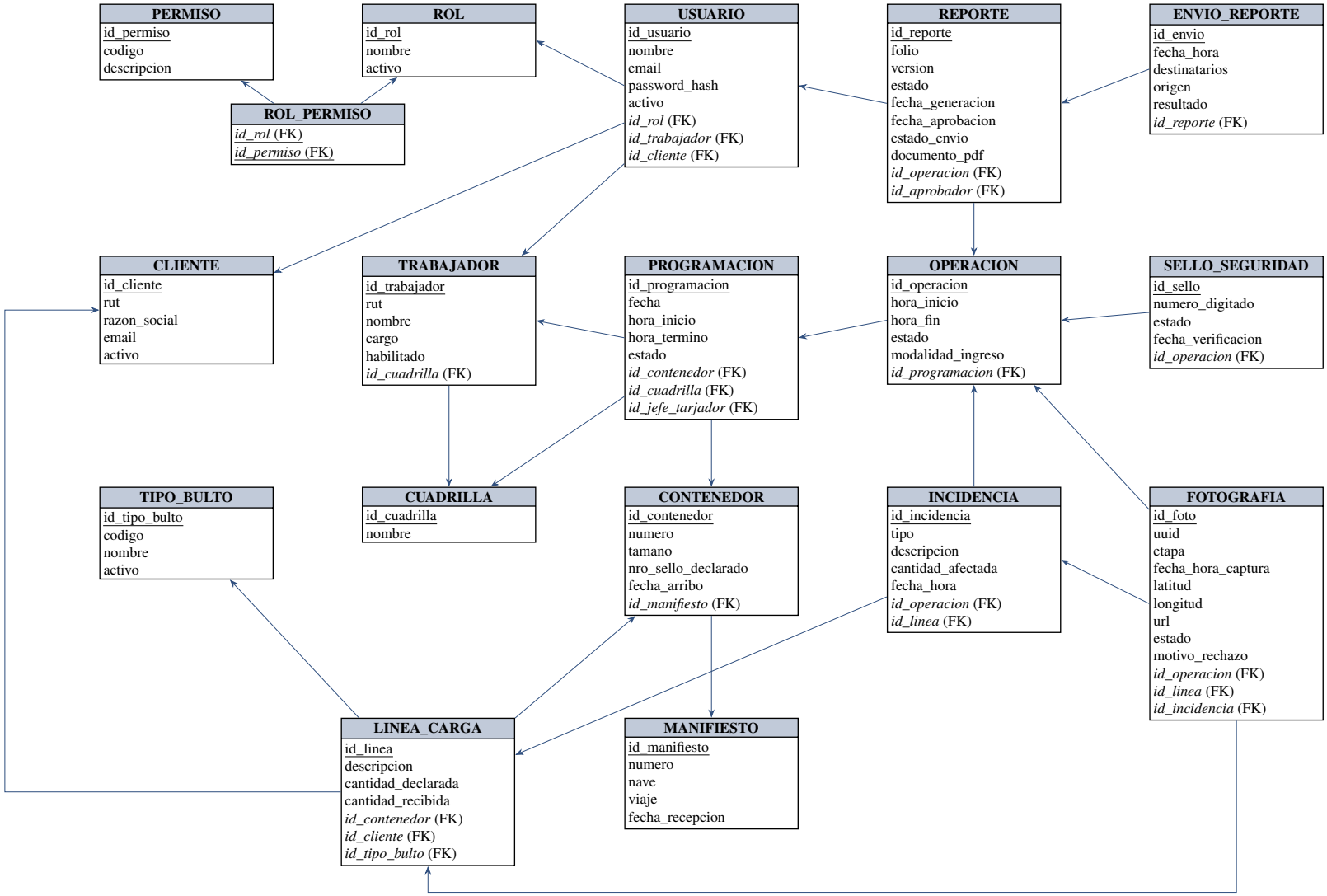


Figura 21: Modelo relacional del sistema BeeTracer.

6.2. Restricciones de integridad

Además de las claves primarias y foráneas, el modelo incorpora las restricciones de la Tabla 6, que llevan a la base de datos las reglas de negocio identificadas en las mini especificaciones.

Tabla 6: Restricciones de integridad del modelo relacional.

Tabla	Restricción
USUARIO	`email` único. Si el rol es de jefe tarjador, `id_trabajador` no es nulo y es único; si es de cliente, `id_cliente` no es nulo.
CLIENTE, TRABAJADOR	`rut` único y con formato válido (dominio `rut`).
PERMISO, TIPO_BULTO	`codigo` único.
MANIFIESTO	Par (`nave`, `viaje`) único: un viaje se importa una sola vez.
CONTENEDOR	`numero` con formato ISO 6346; `tamano` $\in \{20, 40\}$.
LINEA_CARGA	`cantidad_declarada` ≥ 1; `cantidad_recibida` ≥ 0.
PROGRAMACION	`id_contenedor` único (un contenedor tiene una sola programación); `hora_inicio` $<$ `hora_termino`; `estado` $\in$ `estado_faena`.
OPERACION	`id_programacion` único; `hora_fin` no nulo cuando `estado` = FINALIZADA.
SELLO_SEGURIDAD	`id_operacion` único (un sello por operación).
INCIDENCIA	`id_linea` nulo solo si `tipo` = SELLO_ALTERADO; `cantidad_afectada` ≥ 1.
FOTOGRAFIA	`uuid` único; `id_linea` no nulo si `etapa` = LOTE; `id_incidencia` no nulo si `etapa` = INCIDENCIA; `motivo_rechazo` no nulo si `estado` = RECHAZADA.
REPORTE	`folio` único; `id_operacion` único; `version` ≥ 1; `id_aprobador` y `fecha_aprobacion` no nulos si `estado` = APROBADO.
Borrado (todas)	Las claves foráneas usan `ON DELETE RESTRICT`: un registro referenciado no puede eliminarse físicamente, y los datos maestros usados se desactivan (eliminación lógica). La única excepción es `ROL_PERMISO`, que usa `ON DELETE CASCADE` al eliminar un rol sin usuarios.

7. ESPECIFICACIONES DE CADA MÓDULO

Este capítulo especifica los módulos de los diagramas de estructura del Capítulo 5. Mientras las mini especificaciones describen *qué* hace cada proceso del modelo esencial, las especificaciones de módulos describen *cómo* se organiza el programa: quién invoca a cada módulo, qué parámetros recibe y devuelve (sus acoplamientos) y qué pasos ejecuta.

Para cada subsistema se presenta primero una tabla con todos sus módulos y luego la especificación detallada, en pseudocódigo, de sus módulos de control y de los módulos que concentran las reglas de negocio. Las banderas de control se escriben en `monoespaciado`. Los módulos que implementan un proceso elemental siguen la lógica de su mini especificación (Sección 4.3), que se indica entre paréntesis.

7.1. Módulo principal

Tabla 7: Módulos del panorama general.

Módulo	Llamado por	Entradas	Salidas	Función
Sistema BeeTracer	Ninguno (raíz)	Ninguna	Ninguna	Centro de transacción: obtiene la solicitud, la despacha y muestra el resultado.
Obtener transacción	Sistema BeeTracer	Ninguna	solicitud	Lee la solicitud y autentica al usuario.
Leer solicitud	Obtener transacción	Ninguna	solicitud	Captura la solicitud desde la interfaz web o móvil.
Despachar transacción	Sistema BeeTracer	solicitud	resultado	Selecciona el subsistema según el tipo de transacción.
Mostrar resultado	Sistema BeeTracer	resultado	Ninguna	Presenta la respuesta en la interfaz que originó la solicitud.

Módulo: Sistema BeeTracer

Llama a: Obtener transacción, Despachar transacción, Mostrar resultado.

Entradas / salidas: Ninguna (módulo raíz).

Proceso:

1. **REPETIR** mientras el sistema esté en servicio:
   a. Obtener transacción → solicitud, `autenticado`.
   b. **SI** `autenticado` = falso **ENTONCES** resultado = “acceso denegado” **SINO SI** el rol del usuario no tiene el permiso de la transacción **ENTONCES** resultado = “operación no permitida”

SINO Despachar transacción(solicitud) → resultado.

c. Mostrar resultado(resultado).

Módulo: Despachar transacción

Llamado por: Sistema BeeTracer.

Llama a: Los seis subsistemas (selección).

Entradas: solicitud. **Salidas:** resultado.

Proceso:

1. **SEGÚN** el tipo de solicitud:
   a. mantención de datos maestros → Administrar datos maestros.
   b. importación → Importar manifiesto.
   c. programación o consulta de faenas asignadas → Programar faenas.
   d. registro de terreno → Ejecutar operación en terreno.
   e. revisión, decisión o reemplazo de evidencia → Validar reporte.
   f. reenvío o consulta de operaciones y reportes → Distribuir y consultar reportes.
2. **SI** Ejecutar operación en terreno devuelve `finalizada` **ENTONCES** invocar Validar reporte con la transacción "generar borrador".
3. **SI** Validar reporte devuelve `aprobado` **ENTONCES** invocar Distribuir y consultar reportes con la transacción "enviar reporte".
4. Retornar resultado.

7.2. Subsistema 1: Administrar datos maestros

Tabla 8: Módulos del subsistema Administrar datos maestros.

Módulo	Llamado por	Entradas	Salidas	Función
Administrar datos maestros	Despachar transacción	solicitud de mantención	resultado de mantención	Valida y selecciona la mantención según el tipo de dato maestro.
Leer solicitud de mantención	Administrar datos maestros	Ninguna	solicitud	Lee el formulario y la acción solicitada.
Validar datos maestros	Administrar datos maestros	solicitud	`válido`, errores	Verifica campos obligatorios y formatos.
Mantener usuario	Administrar datos maestros	datos de usuario	resultado	Aplica la acción sobre el usuario (mini especificación 1.1).
Mantener rol y permisos	Administrar datos maestros	datos de rol	resultado	Aplica la acción sobre el rol y sus permisos (1.2).
Mantener tipo de bulto	Administrar datos maestros	datos de tipo de bulto	resultado	Aplica la acción sobre el tipo de bulto (1.3).
Mantener cliente	Administrar datos maestros	datos de cliente	resultado	Aplica la acción sobre el cliente (1.4).

Módulo	Llamado por	Entradas	Salidas	Función
Mantener trabajador y cuadrilla	Administrar datos maestros	datos de trabajador	resultado	Aplica la acción sobre el trabajador o la cuadrilla (1.5).
Mostrar resultado de mantención	Administrar datos maestros	resultado	Ninguna	Informa el éxito o los errores al Administrador.

Módulo: Administrar datos maestros

Llamado por: Despachar transacción.

Llama a: Leer solicitud de mantención, Validar datos maestros, los cinco módulos Mantener (selección), Mostrar resultado de mantención.

Entradas: solicitud de mantención. **Salidas:** resultado de mantención.

Proceso:

1. Leer solicitud de mantención → solicitud.
2. Validar datos maestros(solicitud) → `válido`, errores.
3. **SI** `válido` = falso **ENTONCES** resultado = errores **SINO SEGÚN** el tipo de dato maestro invocar Mantener usuario, Mantener rol y permisos, Mantener tipo de bulto, Mantener cliente o Mantener trabajador y cuadrilla → resultado.
4. Mostrar resultado de mantención(resultado).

Módulo: Validar datos maestros

Llamado por: Administrar datos maestros.

Llama a: Validar RUT, Validar email.

Entradas: solicitud. **Salidas:** `válido`, errores.

Proceso:

1. errores = lista vacía.
2. **PARA CADA** campo obligatorio de la acción: **SI** está vacío **ENTONCES** agregar "campo obligatorio" a errores.
3. **SI** la solicitud incluye rut **ENTONCES** Validar RUT(rut) → `rut válido`; **SI** es falso, agregar el error.
4. **SI** la solicitud incluye email **ENTONCES** Validar email(email) → `email válido`; **SI** es falso, agregar el error.
5. `válido` = (errores está vacía); retornar `válido` y errores.

7.3. Subsistema 2: Importar manifiesto

Tabla 9: Módulos del subsistema Importar manifiesto.

Módulo	Llamado por	Entradas	Salidas	Función
Importar manifiesto	Despachar transacción	solicitud de importación	resultado de importación	Coordina las ramas aferente, central y eferente.
Obtener manifiesto válido	Importar manifiesto	Ninguna	manifiesto válido, `válido`, errores	Rama aferente: obtiene y valida el XML.
Leer solicitud de importación	Obtener manifiesto válido	Ninguna	nave, viaje, fecha	Lee los datos del viaje (2.1).
Verificar importación previa	Obtener manifiesto válido	nave, viaje	`duplicado`	Busca un manifiesto del mismo viaje (2.1).
Validar estructura y datos	Obtener manifiesto válido	xml	`válido`, errores	Valida esquema, contenedores y líneas (2.2).
Registrar manifiesto	Importar manifiesto	manifiesto válido	resumen	Centro de transformación: registra cabecera, contenedores y líneas (2.3).
Registrar cabecera	Registrar manifiesto	cabecera	id_manifiesto	Escribe el manifiesto (2.3.1).
Registrar contenedores	Registrar manifiesto	contenedores, id_manifiesto	{id_contenedor}	Escribe los contenedores (2.3.2).
Verificar consignatario	Registrar manifiesto	rut, razon_social	id_cliente, advertencia	Busca o crea el cliente (2.3.3).
Registrar líneas de carga	Registrar manifiesto	líneas	cantidad_lineas	Escribe las líneas (2.3.4).
Emitir resultado de importación	Importar manifiesto	resumen o errores	Ninguna	Rama eferente: informa a Aduanas y al Jefe de faena.
Generar reporte de errores	Emitir resultado de importación	errores	reporte de errores	Compone el reporte (2.4).
Mostrar resumen de importación	Emitir resultado de importación	resumen	Ninguna	Presenta el resumen al Jefe de faena (2.5).

Módulo: Importar manifiesto

Llamado por: Despachar transacción.

Llama a: Obtener manifiesto válido, Registrar manifiesto, Emitir resultado de importación.

Entradas: solicitud de importación. **Salidas:** resultado de importación.

Proceso:

1. Obtener manifiesto válido → manifiesto válido, `válido`, errores.
2. **SI** `válido` **ENTONCES** Registrar manifiesto(manifiesto válido) → resumen; Emitir resultado de

importación(resumen).
3. **SINO** Emitir resultado de importación(errores).

Módulo: Obtener manifiesto válido

Llamado por: Importar manifiesto.

Llama a: Leer solicitud de importación, Verificar importación previa, Descargar XML de Aduanas, Validar estructura y datos.

Entradas: Ninguna. **Salidas:** manifiesto válido, `válido`, errores.

Proceso:

1. Leer solicitud de importación → nave, viaje, fecha, reemplazar.
2. Verificar importación previa(nave, viaje) → `duplicado`.
3. **SI** `duplicado` y no reemplazar **ENTONCES** retornar `válido` = falso con el aviso de importación duplicada.
4. Descargar XML de Aduanas(nave, viaje, fecha) → xml; **SI** el servicio no responde, retornar `válido` = falso con "manifiesto no disponible".
5. Validar estructura y datos(xml) → `válido`, errores; retornar.

7.4. Subsistema 3: Programar faenas

Tabla 10: Módulos del subsistema Programar faenas.

Módulo	Llamado por	Entradas	Salidas	Función
Programar faenas	Despachar transacción	datos de programación o credenciales	confirmación o faenas asignadas	Selecciona la transacción de programación.
Leer solicitud de programación	Programar faenas	Ninguna	solicitud	Lee el formulario o la consulta de la aplicación móvil.
Crear programación	Programar faenas	datos de nueva programación	id_programacion	Crea la programación (3.1).
Modificar programación	Programar faenas	cambio de programación	confirmación	Reprograma, anula, elimina o cambia el estado (3.3).
Verificar conflicto de agenda	Crear y Modificar programación	franja, id_contenedor	`conflicto`	Detecta cruces con otras programaciones.
Asignar cuadrilla y jefe tarjador	Programar faenas	datos de asignación	confirmación	Coordina la asignación (3.2).
Determinar recursos disponibles	Asignar cuadrilla y jefe tarjador	id_programacion	recursos disponibles	Lista cuadrillas y jefes libres (3.2.1).
Verificar disponibilidad	Asignar cuadrilla y jefe tarjador	selección	`disponible`	Revalida la selección (3.2.2).

Módulo	Llamado por	Entradas	Salidas	Función
Registrar asignación	Asignar cuadrilla y jefe tarjador	asignación	Ninguna	Guarda la asignación (3.2.3).
Entregar faenas asignadas	Programar faenas	credenciales	faenas asignadas	Entrega las faenas al jefe tarjador (3.4).
Obtener faenas del jefe tarjador	Entregar faenas asignadas	id_trabajador	faenas	Lee programaciones y carga declarada.
Mostrar confirmación	Programar faenas	confirmación	Ninguna	Presenta el resultado.

Módulo: Asignar cuadrilla y jefe tarjador

Llamado por: Programar faenas.

Llama a: Determinar recursos disponibles, Verificar disponibilidad, Registrar asignación.

Entradas: datos de asignación. **Salidas:** confirmación de programación.

Proceso:

1. **SI** los datos son una solicitud de recursos **ENTONCES** Determinar recursos disponibles(id_programacion) → recursos; retornar recursos.
2. **SINO** Verificar disponibilidad(selección) → `disponible`.
3. **SI** `disponible` **ENTONCES** Registrar asignación(selección) y retornar "asignación registrada" **SINO** retornar "rechazo de asignación" indicando el recurso no disponible.

Módulo: Verificar conflicto de agenda

Llamado por: Crear programación, Modificar programación (fan-in).

Llama a: Acceder a la base de datos.

Entradas: fecha, hora_inicio, hora_termino, id_contenedor. **Salidas:** `conflicto`.

Proceso:

1. Leer las programaciones del mismo contenedor o de la misma fecha que no estén ANULADAS ni FINALIZADAS, excluyendo la que se está modificando.
2. `conflicto` = existe alguna con el mismo contenedor, o alguna cuyo intervalo [hora_inicio, hora_termino) se intersecta con el nuevo y comparte cuadrilla o jefe tarjador.
3. Retornar `conflicto`.

7.5. Subsistema 4: Ejecutar operación en terreno

Este subsistema se ejecuta en la aplicación móvil. A diferencia del modelo esencial, el diseño incorpora el trabajo sin conexión: cada fotografía se comprime, se sella y se encola localmente, y se envía de inmediato si hay señal o al recuperarla.

Tabla 11: Módulos del subsistema Ejecutar operación en terreno.

Módulo	Llamado por	Entradas	Salidas	Función
Ejecutar operación en terreno	Despachar transacción	registro de terreno	confirmación de registro, `finalizada`	Selecciona la etapa del flujo guiado.
Leer registro de terreno	Ejecutar operación en terreno	Ninguna	registro	Lee los datos e imágenes de la etapa actual.
Iniciar operación	Ejecutar operación en terreno	datos de inicio	confirmación	Crea la operación (4.1).
Verificar sello	Ejecutar operación en terreno	datos de sello	estado_sello	Compara el sello y exige incidencia si no está intacto (4.2).
Registrar lote	Ejecutar operación en terreno	datos de lote	confirmación	Identifica la línea y coordina el contraste (4.3).
Contrastar cantidad	Registrar lote	id_linea, cantidad	diferencia	Acumula lo recibido (4.3.2).
Evaluar diferencia	Registrar lote	diferencia, dano_visible	`incidencia requerida`	Decide si se exige incidencia (4.3.3).
Registrar incidencia	Ejecutar operación en terreno	datos de incidencia	id_incidencia	Registra el hallazgo (4.4).
Registrar evidencia fotográfica	Verificar sello, Registrar lote, Registrar incidencia, Cerrar faena	imagen, etapa, referencias	id_foto	Aplica la regla de evidencia obligatoria.
Cerrar faena	Ejecutar operación en terreno	datos de cierre	`finalizada`	Verifica evidencia, registra faltantes y cierra (4.5).
Verificar evidencia obligatoria	Cerrar faena	id_operacion	`completa`, etapas faltantes	Revisa que existan las etapas obligatorias.
Registrar faltantes	Cerrar faena	líneas incompletas	Ninguna	Crea incidencias FALTANTE confirmadas.
Mostrar confirmación	Ejecutar operación en terreno	confirmación	Ninguna	Indica la siguiente etapa del flujo guiado.

Módulo: Registrar evidencia fotográfica

Llamado por: Verificar sello, Registrar lote, Registrar incidencia, Cerrar faena (fan-in).

Llama a: Comprimir fotografía, Sellar fecha, hora y geoposición, Encolar fotografía.

Entradas: imagen, etapa, id_operacion, (id_linea), (id_incidencia). **Salidas:** id_foto.

Proceso:

1. **SI** la imagen está vacía o es ilegible **ENTONCES** retornar error: no se permite avanzar sin una fotografía válida.
2. Comprimir fotografía(imagen) → imagen comprimida.
3. Sellar fecha, hora y geoposición(imagen comprimida) → fotografía con uuid, fecha_hora_captura, latitud y longitud.
4. Guardar la fotografía en la base local con estado = PENDIENTE y sus referencias.
5. Encolar fotografía(fotografía): si hay conectividad se sube de inmediato y pasa a SUBIDA; si no, permanece en la cola.
6. Retornar id_foto.

Módulo: Cerrar faena

Llamado por: Ejecutar operación en terreno.

Llama a: Registrar evidencia fotográfica, Verificar evidencia obligatoria, Registrar faltantes, Sincronizar pendientes.

Entradas: datos de cierre. **Salidas:** confirmación de registro, `finalizada`.

Proceso:

1. Registrar evidencia fotográfica(imagen, etapa = CIERRE).
2. Verificar evidencia obligatoria(id_operacion) → `completa`; **SI** es falsa, retornar las etapas faltantes con `finalizada` = falso.
3. **SI** hay líneas con cantidad_recibida $<$ cantidad_declarada **ENTONCES** pedir confirmación; **SI** se confirma, Registrar faltantes(líneas) **SINO** retornar sin cerrar.
4. Sincronizar pendientes → cantidad enviada; las fotografías que no puedan enviarse se sincronizarán en segundo plano.
5. Registrar hora_fin y estado = FINALIZADA en la operación y en la programación.
6. Retornar `finalizada` = verdadero.

7.6. Subsistema 5: Validar reporte

Tabla 12: Módulos del subsistema Validar reporte.

Módulo	Llamado por	Entradas	Salidas	Función
Validar reporte	Despachar transacción	solicitud de validación	reporte para revisión, `aprobado`	Selecciona generar, presentar, aprobar, rechazar o reemplazar.
Leer solicitud de validación	Validar reporte	Ninguna	solicitud	Lee la acción del Supervisor o del jefe tarjador.
Generar borrador de reporte	Validar reporte	id_operacion	id_reporte	Compone y guarda el reporte (5.1).
Obtener datos de la operación	Generar borrador de reporte	id_operacion	datos	Lee operación, sello, incidencias y líneas.

Módulo	Llamado por	Entradas	Salidas	Función
Ordenar evidencia por etapa y cliente	Generar borrador de reporte	fotografías	fotografías ordenadas	Agrupa por etapa y, en lotes, por cliente.
Presentar reporte	Validar reporte	id_reporte	reporte	Muestra la bandeja o un reporte (5.2).
Aprobar reporte	Validar reporte	id_reporte, id_aprobador	`aprobado`	Aprueba si la evidencia está completa (5.3).
Verificar evidencia obligatoria	Aprobar reporte	id_operacion	`completa`	Revisa etapas y rechazos pendientes.
Registrar aprobación	Aprobar reporte	id_reporte, id_aprobador	Ninguna	Cambia el estado a APROBADO.
Rechazar evidencia	Validar reporte	id_foto, motivo	Ninguna	Rechaza y solicita recaptura (5.4).
Marcar fotografía rechazada	Rechazar evidencia	id_foto, motivo	Ninguna	Guarda RECHAZADA y el motivo.
Reemplazar evidencia	Validar reporte	id_foto, imagen	`corregida`	Registra el reemplazo (5.5).
Registrar fotografía de reemplazo	Reemplazar evidencia	id_foto, imagen	Ninguna	Crea la nueva fotografía y marca la anterior REEMPLAZADA.

Módulo: Validar reporte

Llamado por: Despachar transacción.

Llama a: Leer solicitud de validación y, por selección, Generar borrador de reporte, Presentar reporte, Aprobar reporte, Rechazar evidencia o Reemplazar evidencia.

Entradas: solicitud de validación. **Salidas:** reporte para revisión, `aprobado`.

Proceso:

1. Leer solicitud de validación → solicitud.
2. **SEGÚN** la acción: generar → Generar borrador de reporte; revisar → Presentar reporte; aprobar → Aprobar reporte; rechazar → Rechazar evidencia; reemplazar → Reemplazar evidencia.
3. **SI** Reemplazar evidencia retorna `corregida` **ENTONCES** Generar borrador de reporte(id_operacion) para crear la nueva versión.
4. Retornar el resultado y, si corresponde, `aprobado` = verdadero.

7.7. Subsistema 6: Distribuir y consultar reportes

Tabla 13: Módulos del subsistema Distribuir y consultar reportes.

Módulo	Llamado por	Entradas	Salidas	Función
Distribuir y consultar reportes	Despachar transacción	solicitud	resultado	Selecciona enviar, consultar o entregar.

Módulo	Llamado por	Entradas	Salidas	Función
Leer solicitud	Distribuir y consultar reportes	Ninguna	solicitud	Lee la solicitud o los filtros.
Enviar reporte	Distribuir y consultar reportes	id_reporte	resultado de envío	Envía por correo y registra el envío (6.1).
Obtener destinatarios	Enviar reporte	id_reporte	destinatarios	Correos de los clientes con carga en la operación.
Registrar envío	Enviar reporte	resultado, destinatarios	Ninguna	Escribe el envío y el estado_envio.
Consultar operaciones	Distribuir y consultar reportes	consulta de operaciones	listado de operaciones	Filtra y, si se pide, cambia el estado (6.2).
Modificar estado de operación	Consultar operaciones	id_operacion, nuevo estado	Ninguna	Cambia el estado si es una transición válida.
Filtrar operaciones	Consultar operaciones, Consultar estado del cliente	filtros	operaciones	Aplica los filtros y, para clientes, restringe a su carga.
Consultar estado del cliente	Distribuir y consultar reportes	consulta de estado	estado de operaciones	Atiende la consulta del cliente (6.3).
Entregar reporte PDF	Distribuir y consultar reportes	solicitud de PDF	reporte PDF	Entrega el PDF aprobado (6.4).
Verificar acceso al reporte	Entregar reporte PDF	id_reporte, id_cliente	`permitido`	Comprueba estado APROBADO y pertenencia.
Leer documento PDF	Entregar reporte PDF	id_reporte	pdf	Recupera el documento almacenado.
Mostrar resultado	Distribuir y consultar reportes	resultado	Ninguna	Presenta el listado, el estado o el resultado del envío.

Módulo: Enviar reporte

Llamado por: Distribuir y consultar reportes (envío automático o reenvío).

Llama a: Obtener destinatarios, Enviar correo, Registrar envío.

Entradas: id_reporte, origen. **Salidas:** resultado de envío.

Proceso:

1. **SI** el reporte no está APROBADO **ENTONCES** retornar “reporte pendiente de validación”.
2. Obtener destinatarios(id_reporte) → destinatarios.
3. **SI** destinatarios está vacía **ENTONCES** resultado = FALLIDO, “completar ficha del cliente”.

4. **SINO** Enviar correo(destinatarios, asunto, pdf) → `aceptado`; resultado = ENVIADO **SI** `aceptado` **SINO** FALLIDO.
5. Registrar envío(resultado, destinatarios, origen); retornar resultado.

7.8. Módulos predefinidos

Los módulos predefinidos son servicios de biblioteca o interfaces con sistemas externos que el diseño reutiliza (Tabla 14). Corresponden a las clases de servicio del Informe 2 y concentran la tecnología que el modelo esencial había abstraído.

Tabla 14: Módulos predefinidos del diseño.

Módulo	**Entradas / salidas**	**Función**	**Implementación (Informe 2)**
Autenticar usuario	credenciales / `autenticado`, id_usuario	Verifica email, password_hash y cuenta activa.	`Usuario`: *iniciarSesion()*
Acceder a la base de datos	consulta / registros	Lee y escribe en las tablas del modelo relacional.	Capa de persistencia
Validar RUT, Validar email	valor / `válido`	Verifican formato y dígito verificador.	Biblioteca de validación
Calcular hash de contraseña	password / password_hash	Genera el resumen criptográfico.	`Usuario`: *cambiarPassword()*
Guardar registro maestro	registro / `guardado`	Escribe o actualiza un dato maestro.	Capa de persistencia
Descargar XML de Aduanas	nave, viaje, fecha / xml	Consume el web service del Servicio Nacional de Aduanas.	`ServicioAduana`: *descargarXML()*
Notificar a Aduanas	nave, viaje, resultado / –	Informa el resultado de la importación.	`ServicioAduana`: *notificarResultadoImportacion()*
Comprimir fotografía	imagen / imagen comprimida	Reduce el tamaño antes del envío.	`Fotografia`: *comprimir()*
Sellar fecha, hora y geoposición	imagen / fotografía	Agrega uuid, fecha, hora, latitud y longitud.	Servicios del dispositivo móvil
Encolar fotografía	fotografía / –	Agrega a la cola local y envía si hay señal.	`SincronizadorOffline`: *encolar()*
Sincronizar pendientes	– / cantidad enviada	Envía las fotografías encoladas.	`SincronizadorOffline`: *sincronizarPendientes()*
Generar PDF	contenido / pdf	Compila datos y fotografías en el documento.	`Reporte`: *generarPDF()*
Notificar a terreno	solicitud de recaptura / –	Envía la solicitud a la aplicación del jefe tarjador.	`GestorReportes`: *solicitarCorreccionEvidencia()*
Enviar correo	destinatarios, asunto, pdf / `aceptado`	Entrega el correo al servidor de salida.	`ServicioCorreo`: *enviar()*

8. CONCLUSIONES

El objetivo de este informe fue modelar BeeTracer con el paradigma estructurado, dando continuidad a la reconstrucción por ingeniería inversa iniciada en el Informe 2. El análisis partió del entorno del sistema: la declaración de propósitos fijó su razón de ser, registrar de forma verificable el estado de la carga en cada etapa del desconsolidado, y el diagrama de contexto y la lista de acontecimientos delimitaron los seis terminadores y los veinticuatro estímulos a los que debe responder. Desde allí, los diagramas de flujo de datos descompusieron el sistema en seis procesos principales, veintiocho procesos de nivel 1 y diez de nivel 2, cuyos flujos y almacenes quedaron definidos sin ambigüedad en el diccionario de datos y cuya lógica quedó descrita en las mini especificaciones.

Comparar ambos enfoques sobre el mismo sistema fue la principal lección del trabajo. El modelo orientado a objetos del Informe 2 organizaba el sistema en torno a *quién* es responsable de cada dato y comportamiento; el modelo estructurado lo organiza en torno a *qué* se hace con los datos y dónde quedan. Este cambio de perspectiva hizo visibles detalles que el diagrama de clases no mostraba con claridad: la necesidad de registrar la etapa de cada fotografía para ordenar el reporte, la conveniencia de registrar cada envío del reporte como un hecho propio y la obligación de registrar el contenedor cuando se opera en tarja ciega. A la vez, la trazabilidad entre ambos modelos se mantuvo: las entidades del DER coinciden con las clases de entidad, y los módulos predefinidos del diseño, con las clases de servicio.

La distinción entre el modelo esencial y el diseño resultó especialmente útil en BeeTracer. Al suponer tecnología perfecta, el análisis se concentró en las reglas de negocio (evidencia obligatoria, contraste de cantidades, validación del supervisor) sin distraerse con la falta de señal en el patio. En el diseño, esos aspectos tecnológicos se incorporaron como módulos bien delimitados (compresión, sellado, encolado y sincronización de fotografías), y el análisis de transacción permitió obtener una arquitectura en la que cada subsistema corresponde a un proceso del nivel 0. El módulo *Registrar evidencia fotográfica*, reutilizado por todas las etapas, muestra cómo un buen diseño concentra en un solo lugar la regla que da valor al producto.

Como limitación, al igual que en el informe anterior, no se tuvo acceso al código ni a la base de datos de BeeTracer, por lo que los procesos, módulos y tablas propuestos son una reconstrucción coherente con lo observado y no necesariamente idéntica a la implementación real. Aun así, el modelo relacional y las especificaciones de módulos constituyen una base concreta para una eventual implementación o para evaluar mejoras del sistema, como la auditoría de los cambios de estado o la integración con los módulos complementarios que la empresa comercializa por separado.

9. REFERENCIAS BIBLIOGRÁFICAS

Bilix Ingeniería. (s.f.). *BeeTracer: Gestiona, consulta y traza tus faenas logísticas con facilidad*. Consultado el 22 de septiembre de 2026, desde https://www.beetracer.cl/

Chen, P. P.-S. (1976). The entity-relationship model: Toward a unified view of data. *ACM Transactions on Database Systems*, *1*(1), 9-36. https://doi.org/10.1145/320434.320440

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, *13*(6), 377-387. https://doi.org/10.1145/362384.362685

Contreras, B., Espina, R., Guarda, J., Ruiz, I., & Codoceo, S. (2026). *Informe 2: Modelamiento UML del sistema BeeTracer* (Informe de la asignatura ICI-3242 Modelamiento de Software). Pontificia Universidad Católica de Valparaíso.

DeMarco, T. (1979). *Structured analysis and system specification*. Prentice Hall.

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7ª ed.). Pearson.

Equipo de desarrollo de BeeTracer, Bilix Ingeniería. (2026). Charla técnica sobre el proceso de desconsolidado y la plataforma BeeTracer. *Pontificia Universidad Católica de Valparaíso, Escuela de Ingeniería Informática*.

Larman, C. (2003). *UML y patrones: Una introducción al análisis y diseño orientado a objetos y al proceso unificado* (2ª ed.). Pearson Educación.

Page-Jones, M. (1988). *The practical guide to structured systems design* (2ª ed.). Prentice Hall.

Pressman, R. S., & Maxim, B. R. (2021). *Ingeniería del software: Un enfoque práctico* (9ª ed.). McGraw-Hill.

Sommerville, I. (2011). *Ingeniería de software* (9ª ed.). Pearson Educación.

Tantau, T. (2023). *The TikZ and PGF packages: Manual for version 3.1.10*. https://ctan.org/pkg/pgf

Yourdon, E. (1989). *Modern structured analysis*. Prentice Hall.

Yourdon, E., & Constantine, L. L. (1979). *Structured design: Fundamentals of a discipline of computer program and systems design*. Prentice Hall.